Introduzione

L'intelligenza artificiale trova spazio in attività quotidiane e professionali, offrendo strumenti con cui elaborare informazioni e affrontare compiti differenti. Tra questi, i modelli linguistici di grandi dimensioni, indicati con l'acronimo LLM (*Large Language Models*), permettono di interagire attraverso il linguaggio naturale e possono affiancare l'utente nella scrittura, nella sintesi di testi e nell'organizzazione delle idee [13]. Il loro interesse risiede anche nella possibilità di rendere accessibili queste funzioni mediante un'interazione diretta, nella quale una richiesta può essere precisata e sviluppata nel corso del dialogo.

Il valore di tali strumenti dipende dal modo in cui vengono utilizzati e dalla loro adeguatezza al compito. Le risposte devono essere esaminate alla luce delle informazioni disponibili e delle esigenze dell'attività svolta. L'impiego dell'intelligenza artificiale come supporto richiede quindi di comprenderne le capacità e i limiti, mantenendo un ruolo attivo nell'interpretazione dei risultati.

Le applicazioni dell'intelligenza artificiale comprendono anche l'analisi dei dati visivi. La visione artificiale sviluppa metodi per estrarre informazioni dalle immagini e riconoscere caratteristiche, oggetti o variazioni del loro aspetto. Nell'ambito industriale, queste capacità possono supportare il controllo di qualità attraverso l'identificazione di elementi che si discostano dalle caratteristiche attese di un prodotto [25, 7].

La varietà dei materiali rende questo compito articolato. Una superficie può presentare una texture regolare oppure variazioni naturali di colore e tessitura; un oggetto può richiedere di considerare anche la forma e la disposizione delle sue parti. Le caratteristiche che definiscono la conformità dipendono pertanto dal materiale e dal prodotto osservato. Anche le anomalie assumono forme differenti: possono riguardare alterazioni cromatiche, tagli, pieghe, perforazioni o altri cambiamenti locali, con dimensioni e posizioni variabili. Distinguere un difetto da una variazione ammessa richiede una descrizione della normalità coerente con il contesto di osservazione [7].

Un'ulteriore difficoltà riguarda la disponibilità di esempi. Quando le anomalie sono poco frequenti, raccogliere immagini che rappresentino tutte le possibili tipologie di difetto può risultare difficile. Inoltre, i campioni disponibili non esauriscono necessariamente le manifestazioni che potranno presentarsi in seguito. Questa condizione motiva l'interesse per metodi che apprendono le caratteristiche dei campioni normali e individuano, nelle nuove immagini, deviazioni rispetto alle regolarità apprese [18, 25].

Da queste considerazioni prende forma il mio project work, dedicato al rilevamento delle anomalie visive nell'ispezione industriale. Ho scelto di confrontare quattro modelli, CCW-SSIM, PaDiM, RD4AD e FastFlow, che descrivono la normalità attraverso approcci differenti. Per la sperimentazione ho utilizzato la categoria `leather` del dataset MVTec AD come caso di studio comune. La varietà dei materiali resta il contesto del problema, mentre concentrarsi su un unico materiale permette di esaminare il comportamento delle pipeline nelle condizioni adottate.

La domanda da cui parte il confronto è quanto questi approcci riescano a riconoscere le immagini difettose e a localizzare le regioni anomale. Per affrontarla, alle metriche ho affiancato la lettura delle mappe prodotte: i risultati numerici acquistano significato anche attraverso le risposte che si possono osservare nelle immagini. Il capitolo seguente introduce i concetti necessari a inquadrare il problema e i criteri di valutazione del confronto.

Capitolo 1

Il rilevamento delle anomalie: definizione del problema

Le difficoltà dell'ispezione visiva richiamate nell'introduzione possono essere inquadrate attraverso il problema del rilevamento delle anomalie. L'**anomaly detection**, o rilevamento delle anomalie, consiste nell'identificare osservazioni che si discostano significativamente dal comportamento prevalente dei dati. Tali deviazioni possono suggerire la presenza di un meccanismo generativo differente e, nei contesti applicativi, segnalare eventi che richiedono un approfondimento [18].

La difficoltà del problema dipende soprattutto da tre caratteristiche:

- **scarsa conoscenza preliminare**: alcune tipologie di anomalia diventano note soltanto quando si manifestano;
- **eterogeneità**: anomalie diverse possono presentare caratteristiche visive o statistiche molto differenti;
- **rarità**: i campioni anomali sono spesso poco numerosi rispetto a quelli che descrivono il normale funzionamento.

Queste caratteristiche rendono difficile raccogliere esempi rappresentativi di tutti i difetti possibili. La ricerca sul rilevamento delle anomalie ha quindi sviluppato metodi capaci di apprendere la regolarità dei dati e di riconoscere deviazioni non necessariamente osservate durante l'addestramento. Tra le principali questioni aperte si distinguono [18]:

1. **Sensibilità e falsi allarmi**. Un basso richiamo (*recall*) indica che una parte delle anomalie non viene rilevata, producendo falsi negativi. Aumentare la sensibilità può però incrementare i falsi positivi, ossia i campioni normali segnalati come anomali. Il compromesso tra questi due errori dipende dall'applicazione.

2. **Alta dimensionalità e dipendenze tra osservazioni**. Le deviazioni possono riguardare poche componenti di dati molto complessi. Inoltre, alcune anomalie sono riconoscibili soltanto considerando relazioni spaziali, temporali o contestuali.

3. **Disponibilità limitata di dati**. La raccolta e l'annotazione dei difetti possono essere costose. Occorre quindi sfruttare efficacemente i campioni normali e, quando disponibili, le informazioni parzialmente etichettate.

4. **Robustezza al rumore**. Etichette errate, disturbi di acquisizione e campioni anomali non riconosciuti possono compromettere l'apprendimento della normalità. La tolleranza a tali contaminazioni deve essere valutata per ciascun metodo.

5. **Anomalie complesse**. Oltre alle deviazioni di singole osservazioni, occorre considerare anomalie contestuali, che dipendono dalle condizioni di osservazione, e collettive, che emergono dal comportamento di un gruppo di dati.

6. **Interpretabilità**. Nei domini critici è utile comprendere quali informazioni abbiano determinato una segnalazione. La localizzazione di un difetto può supportare l'ispezione umana, ma non costituisce di per sé una spiegazione del comportamento del modello.

Il deep learning consente di apprendere rappresentazioni dei dati senza definire manualmente tutte le caratteristiche rilevanti. Questo può migliorare il rilevamento in scenari complessi; non elimina, tuttavia, i problemi di robustezza, generalizzazione e interpretabilità. In questo lavoro ho circoscritto l'analisi alle anomalie visive nell'ispezione industriale.

1.1 Il rilevamento delle anomalie nella visione artificiale

La *Visual Anomaly Detection* (VAD) identifica, in dati visivi quali immagini o nuvole di punti, elementi che si discostano da una definizione di normalità [6]. Rispetto alla classificazione in cui le classi sono note a priori, il problema richiede di riconoscere anche difetti assenti dai dati di addestramento [25].

Si distinguono due obiettivi:

- **rilevamento a livello di immagine** (*image-level*): stabilire se un'immagine contenga un'anomalia, mediante un punteggio globale;

- **localizzazione a livello di pixel** (*pixel-level*): assegnare punteggi alle posizioni dell'immagine per individuare le regioni potenzialmente difettose.

Una buona discriminazione globale non implica necessariamente una localizzazione accurata. Un modello può segnalare correttamente un'immagine difettosa e produrre, allo stesso tempo, una mappa troppo estesa o incompleta. La distinzione tra i due obiettivi guida la scelta delle metriche e l'interpretazione dei risultati [25, 18].

1.1.1 Sfide dei contesti industriali

L'integrazione di sistemi di visione e sensori nei processi produttivi offre nuove possibilità di controllo automatizzato. Il trasferimento dei metodi dagli scenari di laboratorio alle applicazioni reali richiede però di affrontare lo sbilanciamento dei dati, la variabilità dei difetti e le condizioni di acquisizione [7, 15].

Sbilanciamento delle classi

Nelle linee di produzione efficienti, i difetti possono essere sporadici. Un classificatore addestrato su dati fortemente sbilanciati rischia di privilegiare la classe normale, trascurando eventi rari ma rilevanti. La scarsità di esempi difettosi motiva l'impiego di approcci che apprendono soprattutto dai campioni conformi [15, 19].

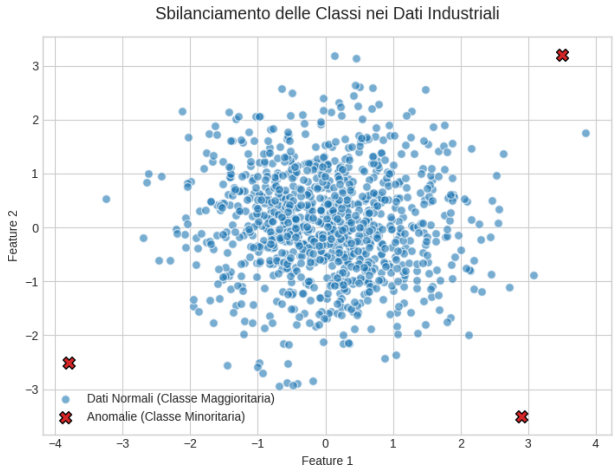


Figura 1.1: Rappresentazione schematica dello sbilanciamento tra campioni normali ed eventi anomali.

Variabilità ed eterogeneità delle anomalie

I difetti possono differire per forma, dimensione, posizione e contrasto rispetto al materiale. Non è quindi realistico assumere di poter rappresentare tutte le

loro configurazioni nell'insieme di addestramento. Il modello deve distinguere le variazioni compatibili con il prodotto conforme dalle deviazioni significative, generalizzando oltre gli esempi osservati [19, 7].

La Figura 1.2 illustra, mediante una serie temporale, la distinzione generale tra anomalie puntuali, contestuali e collettive. Nell'ispezione visiva, l'eterogeneità si manifesta invece attraverso alterazioni cromatiche, discontinuità della tessitura (*texture*) o modifiche della struttura dell'oggetto.

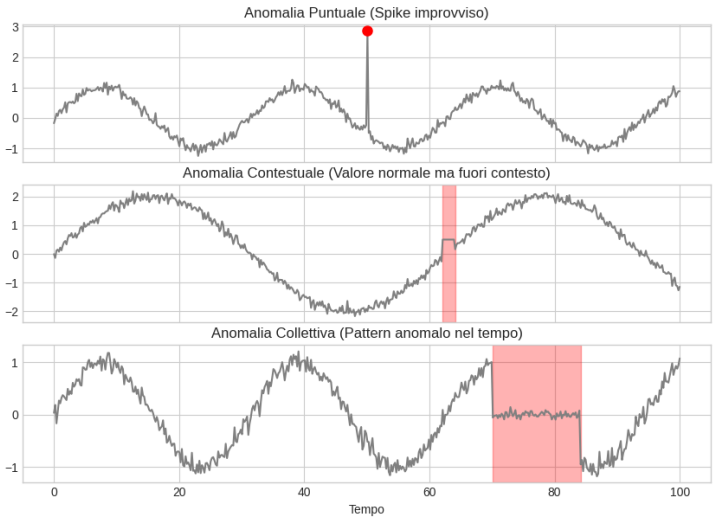


Figura 1.2: Esempio delle tipologie generali di anomalia in una serie temporale: puntuale, contestuale e collettiva.

Dipendenza dal contesto e condizioni di acquisizione

La normalità dipende dalle condizioni operative. Un valore di temperatura può essere accettabile in una fase produttiva e indicare un guasto in un'altra; analogamente, nell'ispezione visiva, illuminazione, riflessi e variazioni ammesse del materiale possono modificare l'aspetto di un prodotto conforme.

I confini tra variazione normale e difetto possono dunque essere ambigui. Il preprocessing e l'eventuale impiego di informazioni contestuali devono essere progettati in funzione del dominio, per limitare le segnalazioni dovute ai disturbi di acquisizione [15, 1]. La Figura 1.3 esemplifica il ruolo del contesto nella scelta di una soglia di rilevamento.

1.1.2 Paradigmi di apprendimento e convenzione terminologica

La disponibilità delle etichette determina quali informazioni possano guidare l'apprendimento. È utile distinguere il paradigma generale dalla granularità

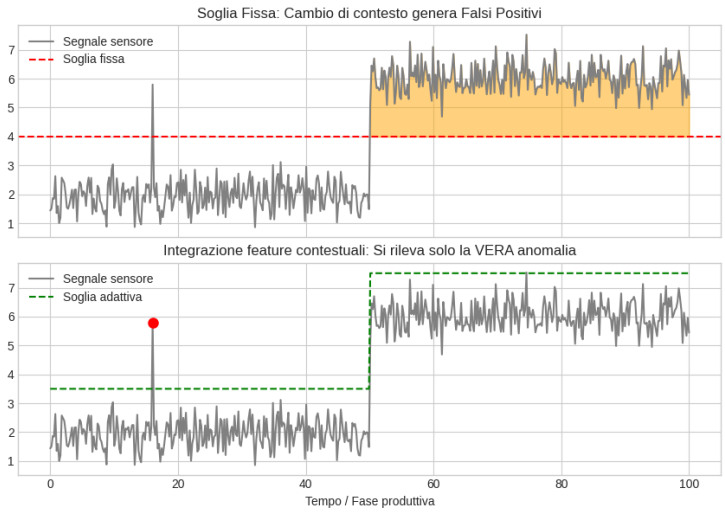


Figura 1.3: Esempio schematico di soglia fissa e soglia adattiva al cambiamento della fase produttiva.

delle annotazioni e dalla specifica composizione dell'insieme di addestramento [25].

Apprendimento supervisionato

Nell'apprendimento supervisionato, ciascun campione è associato a un'etichetta utilizzata durante l'addestramento. Per il rilevamento dei difetti, le etichette possono indicare la conformità dell'intera immagine oppure descrivere le regioni difettose mediante maschere a livello di pixel.

Le annotazioni spaziali richiedono un maggiore lavoro manuale. Le etichette riferite alla sola immagine costituiscono invece una forma di supervisione debole quando l'obiettivo finale è localizzare il difetto. Il vantaggio della supervisione consiste nel fornire esempi espliciti delle anomalie; il limite principale è la necessità di raccogliere campioni sufficientemente rappresentativi [2].

Quando i difetti disponibili sono pochi o poco diversificati, il modello può adattarsi eccessivamente agli esempi osservati e riconoscere con difficoltà anomalie nuove. Questo rischio non implica che tutti i metodi supervisionati falliscano sui difetti sconosciuti, ma rende necessario verificarne la capacità di generalizzazione.

Apprendimento non supervisionato in senso generale

Nell'apprendimento non supervisionato non sono disponibili etichette che distinguano i campioni normali da quelli anomali. Molti metodi assumono che le anomalie siano rare e statisticamente distinguibili dalla maggioranza dei dati;

il rilevamento dipende quindi dalla rappresentazione adottata e dalla misura di deviazione [18].

L'assenza di annotazioni riduce il costo di preparazione dei dati, ma non garantisce robustezza alla contaminazione. Se le anomalie influenzano il modello della normalità, possono diventare più difficili da riconoscere. Esistono metodi progettati per affrontare questo problema, la cui efficacia dipende dalle ipotesi e dalle condizioni sperimentali [14].

Apprendimento dai soli campioni normali

Una terza impostazione utilizza un insieme di addestramento composto esclusivamente da campioni noti come normali, senza esempi delle anomalie da rilevare. Nella tassonomia generale dell'anomaly detection, questa impostazione è spesso denominata *semi-supervisionata* [23]. I metodi costruiscono una descrizione della classe normale attraverso confini decisionali, distribuzioni statistiche o rappresentazioni neurali.

Nella letteratura sull'ispezione visiva industriale, lo stesso scenario è comunemente indicato come *unsupervised anomaly detection*, poiché l'apprendimento non richiede esempi annotati dei difetti [7]. Per evitare ambiguità, nell'elaborato adotto l'espressione **scenario one-class**: l'adattamento al dominio avviene sui soli campioni normali, mentre le etichette e le maschere del test sono riservate alla valutazione.

Questa convenzione si applica a tutti i modelli selezionati. L'eventuale uso di una rete che estrae vettori di caratteristiche (*feature*), detta *backbone*, preaddestrato su ImageNet riguarda la provenienza della rappresentazione iniziale e deve essere distinto dalle informazioni impiegate nell'adattamento al dominio industriale.

1.2 Il benchmark MVTec AD e le metriche di valutazione

Un benchmark condiviso permette di confrontare metodi differenti sugli stessi dati e con criteri espliciti. MVTec Anomaly Detection (**MVTec AD**) è uno dei riferimenti per l'ispezione visiva industriale: comprende campioni normali per l'addestramento e immagini sia normali sia difettose per il test, con annotazioni delle anomalie a livello di pixel [4].

1.2.1 Texture e oggetti strutturati

MVTec AD comprende 5354 immagini ad alta risoluzione, suddivise in quindici categorie:

- **cinque categorie di texture**: tappeto, griglia, pelle, piastrelle e legno;
- **dieci categorie di oggetti**: bottiglia, cavo, capsula, nocciola, dado metallico, pillola, vite, spazzolino, transistor e cerniera.

Le texture richiedono soprattutto di riconoscere alterazioni delle regolarità locali; gli oggetti introducono anche vincoli relativi a forma, posizione e composizione delle parti. Il dataset contiene oltre settanta tipologie di difetto, tra cui graffi, contaminazioni, deformazioni e componenti mancanti [4].

La sperimentazione documentata nel presente elaborato riguarda la categoria `leather`, appartenente alle texture. La descrizione delle altre categorie fornisce il contesto del benchmark, ma non implica che siano incluse negli esperimenti.

1.2.2 Estensioni del benchmark

L'evoluzione dei metodi ha motivato la costruzione di dataset complementari. **MVTec LOCO AD** (*Logical Constraints Anomaly Detection*) comprende 3644 immagini e considera sia anomalie strutturali sia violazioni di vincoli logici, come componenti corretti collocati nella posizione sbagliata [17].

MVTec AD 2 amplia invece la varietà degli scenari industriali con oltre 8000 immagini ad alta risoluzione. Include oggetti sovrapposti, materiali trasparenti o semitrasparenti, variazioni di acquisizione e difetti di piccole dimensioni rispetto all'immagine [12]. Queste condizioni mettono alla prova capacità che possono essere poco sollecitate dai benchmark precedenti.

I dataset citati in questa sezione inquadrano le sfide della ricerca; non costituiscono dati sperimentali del confronto descritto nel Capitolo 3.

1.2.3 Criteri di valutazione

Il confronto distingue la discriminazione dell'intera immagine dalla localizzazione del difetto. Le metriche basate su curve valutano il comportamento al variare della soglia, mentre quelle calcolate su una singola segmentazione dipendono dalla soglia scelta [28].

AUROC a livello di immagine

La curva ROC rappresenta il tasso di veri positivi (*true positive rate*, TPR) in funzione del tasso di falsi positivi (*false positive rate*, FPR):

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad \mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}.$$

Le anomalie costituiscono la classe positiva. L'area sotto la curva, denominata AUROC, misura la capacità dei punteggi di discriminare campioni normali e difettosi al variare della soglia.

A livello di immagine, il calcolo richiede un punteggio globale per ciascun campione. Nel protocollo adottato, tale punteggio è il massimo della mappa di anomalia. Un'AUROC elevata indica una buona separazione dei campioni nel test esaminato, ma non determina una soglia operativa né garantisce la stessa prestazione su dati futuri.

AUROC a livello di pixel

L'AUROC a livello di pixel confronta i punteggi della mappa con le maschere di riferimento (*ground truth*). La metrica valuta l'ordinamento dei pixel normali e anomali, ma non descrive direttamente la precisione dei contorni o la copertura di ciascun difetto.

Poiché TPR e FPR sono normalizzati sulle rispettive classi, il predominio numerico dei pixel normali non equivale, da solo, a un aumento dell'AUROC. Rimangono però limiti interpretativi: il raggruppamento dei pixel attribuisce maggiore peso alle anomalie estese e l'integrazione sull'intera curva può nascondere un comportamento insoddisfacente ai bassi FPR rilevanti per l'ispezione industriale [5]. Per questo l'AUROC deve essere affiancata da misure complementari e dall'esame delle mappe.

PRO e AU-PRO

Il *Per-Region Overlap* (PRO) misura, a una data soglia, la frazione di pixel rilevati all'interno di ciascuna regione anomala connessa e ne calcola la media. Le regioni ricevono così lo stesso peso, indipendentemente dalla loro area. I falsi positivi esterni ai difetti sono considerati attraverso il FPR associato alla soglia [28].

L'AU-PRO integra il PRO in funzione del FPR entro un intervallo dichiarato. Nel protocollo del progetto, il limite superiore è 0,3 e l'area è normalizzata rispetto a tale intervallo. L'indicazione del limite e della normalizzazione è necessaria per interpretare e confrontare i valori.

Metriche a soglia e analisi qualitativa

Il punteggio F1 combina precisione e richiamo:

$$\mathrm{F1} = \frac{2\,\mathrm{precision}\,\mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}.$$

Nei risultati del progetto, il massimo F1 è ottenuto scegliendo la soglia sull'insieme di test e costituisce quindi una misura retrospettiva *oracle*. Non rappresenta una prestazione operativa con soglia selezionata su dati indipendenti. Il calcolo a livello di pixel è inoltre approssimato mediante campionamento spaziale con passo 4 e una griglia di 500 soglie.

Per interpretare questi indicatori, nel confronto considero anche immagini, maschere e mappe. La lettura qualitativa permette di osservare localizzazioni incomplete, risposte diffuse e attivazioni su regioni normali. Nel discuterle, è necessario distinguere ciò che è visibile dalle ipotesi sulle sue cause.

1.3 Obiettivi e struttura dell'elaborato

Il confronto sviluppato nel progetto riguarda quattro pipeline, cioè procedure che comprendono elaborazione delle immagini, adattamento del modello e

inferenza, per il rilevamento delle anomalie visive industriali: **CCW-SSIM**, **PaDiM**, **RD4AD** e **FastFlow**. I modelli rappresentano approcci differenti alla costruzione della normalità: ricostruzione dell'immagine, modellazione statistica delle feature, distillazione della conoscenza e flussi di normalizzazione.

Ho mantenuto la categoria `leather` di MVTec AD come unico caso di studio, utilizzando dati e protocollo di valutazione condivisi per le implementazioni e le configurazioni adottate. Nella lettura dei risultati considero anche le differenze di risoluzione, pre-addestramento e regime di apprendimento: il confronto comprende queste condizioni e non isola l'effetto della sola famiglia metodologica.

Su questa base, l'obiettivo è valutare la discriminazione delle immagini e la localizzazione dei difetti, esaminare qualitativamente le mappe e discutere i costi osservati e i limiti delle pipeline. L'estensione ad altre categorie e le prove dedicate a colore, illuminazione o adattamento dei pesi pre-addestrati (*fine-tuning*) restano possibili approfondimenti, distinti dagli esperimenti già documentati.

Il Capitolo 2 presenta lo stato dell'arte e i principi dei metodi selezionati. Il Capitolo 3 descrive le implementazioni e il protocollo comune. Il Capitolo 4 analizza quantitativamente e qualitativamente i risultati, mettendoli in relazione con le fonti di riferimento. Il Capitolo 5 sintetizza quanto emerso nel caso di studio e discute i limiti della ricerca e le possibili estensioni.

Capitolo 2

Stato dell'arte

Definiti il problema e i criteri di valutazione, occorre esaminare i metodi con cui apprendere la normalità visiva e riconoscerne le deviazioni. Nell'ispezione industriale, l'obiettivo è riconoscere difetti anche quando non sono disponibili esempi rappresentativi di tutte le anomalie possibili [25, 7].

Le applicazioni comprendono il controllo di qualità, l'analisi di immagini mediche e il rilevamento di eventi nei video. Questi ambiti condividono il problema della scarsità delle anomalie, ma differiscono per dati, vincoli e modalità di valutazione. Nella rassegna ho privilegiato l'ispezione di immagini industriali, per mantenere il riferimento al dominio della sperimentazione.

2.1 Criteri di classificazione dei metodi

La letteratura può essere organizzata secondo tre assi distinti:

- **dominio dei dati**: immagini, video o rappresentazioni tridimensionali;
- **informazioni disponibili durante l'apprendimento**: campioni normali, esempi di anomalie e annotazioni di diversa granularità;
- **meccanismo di rilevamento**: ricostruzione, distanze nello spazio delle feature, stima di densità o discriminazione appresa.

Questi criteri non sono intercambiabili. Un'architettura non determina da sola il paradigma di supervisione e uno stesso metodo può combinare più meccanismi. La tassonomia serve quindi a chiarire il ruolo delle componenti, evitando di considerare le famiglie come categorie necessariamente disgiunte [7].

2.1.1 Granularità dell'output e supervisione

Il rilevamento a livello di immagine produce un punteggio per l'intera immagine; la localizzazione a livello di pixel produce una mappa spaziale. Una segmentazione binaria può essere ottenuta applicando una soglia alla mappa. L'esistenza

di un output spaziale non implica che il modello sia stato addestrato usando maschere dei difetti.

Come definito nel Capitolo 1, il progetto adotta uno scenario *one-class*: i campioni industriali utilizzati per l'adattamento sono tutti normali. Le maschere del test sono impiegate nella valutazione. I modelli possono comunque utilizzare rappresentazioni pre-addestrate su altri dataset. Altri metodi one-class introducono anomalie sintetiche per costruire compiti auto-supervisionati; questa procedura non è utilizzata dalle quattro pipeline sperimentate.

La Tabella 2.1 distingue le principali informazioni che possono guidare l'apprendimento.

Tabella 2.1: Informazioni di addestramento e ruolo delle annotazioni nell'ispezione visiva.

Impostazione	**Dati disponibili**	**Informazione utilizzata**
Supervisionata	Campioni normali e difettosi	Etichette di classe o maschere dei difetti
Debolmente supervisionata	Annotazioni meno dettagliate dell'output richiesto	Ad esempio, etichette di immagine per localizzare un difetto
One-class	Campioni industriali noti come normali	Modellazione della normalità senza esempi reali dei difetti
Non supervisionata in senso generale	Campioni non etichettati, eventualmente contaminati	Regolarità statistiche dei dati e ipotesi sulle anomalie

2.2 Metodi basati su descrittori manuali

Prima della diffusione delle reti profonde, molte pipeline combinavano descrittori progettati manualmente (*handcrafted features*) con classificatori o stimatori statistici [25]. L'efficacia dipendeva dalla capacità dei descrittori di rappresentare le variazioni rilevanti per il dominio.

Tra gli strumenti utilizzati si distinguono:

- **descrittori locali**: SIFT (*Scale-Invariant Feature Transform*) per punti di interesse invarianti alla scala, HOG (*Histograms of Oriented Gradients*) per orientazioni dei gradienti e LBP (*Local Binary Patterns*) per regolarità locali della texture [25, 7];
- **filtri e trasformate**: operatori di gradiente, rilevamento dei bordi, filtri di Gabor e trasformate di Fourier o wavelet per descrivere frequenze, orientazioni e discontinuità [7].

Le rappresentazioni potevano essere utilizzate da tre principali classi di metodi decisionali:

1. **Stima di densità**. Modelli a mistura gaussiana (GMM) e stimatori di densità basati su kernel (KDE) descrivono la distribuzione dei campioni. Una densità bassa rispetto al modello può segnalare un'anomalia.

2. **Classificazione one-class**. One-Class SVM (OCSVM) e Support Vector Data Description (SVDD) costruiscono un confine attorno ai campioni normali nello spazio delle feature.

3. **Modelli spaziali e temporali**. I campi aleatori di Markov (MRF) e altri modelli contestuali considerano dipendenze tra osservazioni. Nell'analisi video possono essere affiancati da descrittori del movimento [22].

L'alta dimensionalità e le variazioni di illuminazione o texture possono rendere difficile la scelta delle feature. Inoltre, l'estrazione e la decisione sono spesso ottimizzate separatamente. Questi limiti hanno motivato lo sviluppo di rappresentazioni apprese; i metodi tradizionali rimangono tuttavia utili quando i dati sono limitati o i vincoli computazionali favoriscono soluzioni semplici [25].

2.3 Famiglie di metodi basati sul deep learning

Le reti profonde permettono di apprendere descrittori multilivello e di combinare informazione locale e rappresentazioni più astratte. Alcuni metodi addestrano una rete sul dominio industriale; altri utilizzano un estrattore pre-addestrato e stimano soltanto un modello delle feature normali.

Per inquadrare i quattro modelli selezionati, le sezioni seguenti presentano cinque famiglie. Nella lettura della letteratura, l'attenzione è rivolta sia ai principi condivisi sia alle differenze da considerare nel confronto sperimentale [7].

2.3.1 Metodi basati sulla ricostruzione

I metodi ricostruttivi apprendono a riprodurre immagini normali attraverso una rappresentazione latente. L'ipotesi è che i difetti, non osservati durante l'addestramento, siano ricostruiti meno fedelmente e producano una discrepanza maggiore [25]. Questa proprietà deve essere verificata: una rete sufficientemente flessibile può ricostruire anche anomalie.

Negli autoencoder, l'encoder comprime l'input e il decoder ne genera una ricostruzione. La funzione obiettivo può misurare differenze tra pixel o utilizzare indici strutturali. Una possibile combinazione è

$$\mathcal{L}_{\mathrm{rec}}(x,\widehat{x}) = (1-\alpha)\|x-\widehat{x}\|_2 + \alpha\big(1-\mathrm{SSIM}(x,\widehat{x})\big), \qquad 0 \le \alpha \le 1.$$

La formula illustra un criterio ricostruttivo e non una perdita universale della famiglia. Il calcolo di una mappa locale richiede inoltre di specificare come le discrepanze siano valutate nello spazio.

Nei modelli generativi come AnoGAN, la ricerca della ricostruzione più vicina può richiedere un'ottimizzazione del vettore latente. Il criterio combina un residuo dell'immagine con una discrepanza tra feature del discriminatore:

$$\mathcal{L}_{\mathrm{AnoGAN}}(z) = (1-\lambda)\|x-G(z)\|_1 + \lambda\|D_{\mathrm{feat}}(x) - D_{\mathrm{feat}}(G(z))\|_1.$$

Qui G è il generatore e D_{feat} indica una rappresentazione intermedia del discriminatore [21].

Tra gli sviluppi della famiglia, CAVGA introduce vincoli di attenzione, mentre DFR (*Deep Feature Reconstruction*) ricostruisce rappresentazioni estratte da una rete pre-addestrata anziché direttamente i pixel. La ricostruzione può quindi operare in spazi differenti e sovrapporsi ad altri approcci della tassonomia [7].

CCW-SSIM, selezionato per il progetto, appartiene a questa famiglia: combina la ricostruzione mediante autoencoder con una valutazione della discrepanza strutturale e cromatica [3]. Per il confronto è rilevante che apprenda direttamente dai campioni industriali senza utilizzare un backbone ImageNet.

2.3.2 Metodi basati sulle rappresentazioni

Questi metodi descrivono i campioni mediante vettori di caratteristiche, indicati come *feature*, spesso estratte da reti pre-addestrate. Il punteggio di anomalia deriva dal confronto con feature normali conservate in memoria oppure con statistiche stimate su di esse [7].

La famiglia comprende approcci con estrattore congelato e varianti che adattano le rappresentazioni al dominio. Non è quindi corretto attribuire a tutti l'assenza di addestramento neurale.

PaDiM (*Patch Distribution Modeling*) stima una gaussiana multivariata per ciascuna posizione spaziale. Il punteggio locale usa la distanza di Mahalanobis:

$$D_M(z) = \sqrt{(z-\mu)^{\mathrm{T}}\Sigma^{-1}(z-\mu)},$$

dove z è il vettore di feature, μ la media locale e Σ la covarianza stimata sui campioni normali. La distanza considera variabilità e correlazioni tra componenti, a differenza della distanza euclidea [8].

Altri esempi sono:

- **SPADE**: confronta feature multiscala attraverso ricerche dei vicini più prossimi; la consultazione dei campioni di riferimento contribuisce al costo di inferenza [25];
- **PatchCore**: conserva feature normali in una *memory bank*, riducendola mediante selezione di un sottoinsieme rappresentativo (*coreset*) [20];
- **CFA** (*Coupled-hypersphere-based Feature Adaptation*): adatta le feature al dominio attraverso vincoli geometrici [7].

Il vantaggio dell'estrattore pre-addestrato è la disponibilità di rappresentazioni già strutturate. Il limite è la possibile distanza tra il dominio del pre-addestramento e le caratteristiche dei materiali industriali. Memoria, costo di adattamento e inferenza dipendono poi dalla specifica modalità di confronto.

2.3.3 Metodi basati sulla distillazione Teacher–Student

La distillazione utilizza una rete *Teacher* per fornire rappresentazioni di riferimento e una rete *Student* addestrata a riprodurle sui campioni normali. In inferenza, la discrepanza tra le rappresentazioni può segnalare una deviazione dalla normalità [7].

Indicando con $f_T^k(x)$ e $f_S^k(x)$ le feature corrispondenti al livello k, una forma generale dell'obiettivo è

$$\mathcal{L}_{\text{dist}} = \sum_k d\big(f_T^k(x), f_S^k(x)\big),$$

dove la distanza d e la modalità di aggregazione dipendono dal metodo. Il punteggio spaziale in inferenza può utilizzare un'aggregazione diversa da quella impiegata nell'addestramento.

RD4AD adotta la *reverse distillation*: lo Student è un decoder che riceve una rappresentazione compressa delle feature del Teacher e tenta di ricostruirle. Un *One-Class Bottleneck Embedding* (OCBE) aggrega le informazioni multiscala. La struttura asimmetrica mira a limitare il trasferimento indiscriminato delle informazioni anomale allo Student [10].

Il metodo ricostruisce dunque feature anziché immagini. Questa relazione con gli approcci ricostruttivi chiarisce perché la tassonomia sia uno strumento descrittivo e non una separazione rigida. Nel progetto, RD4AD permette di confrontare la ricostruzione distillata delle feature con la modellazione statistica di PaDiM.

2.3.4 Metodi basati sui flussi di normalizzazione

I *normalizing flows* apprendono trasformazioni invertibili tra una distribuzione complessa e una distribuzione di riferimento trattabile. Nell'ispezione visiva possono operare sulle feature estratte da un backbone [7].

Se $z = f_\theta(x)$ è una trasformazione invertibile, il cambio di variabile fornisce

$$p_X(x) = p_Z(f_\theta(x)) \left| \det \frac{\partial f_\theta(x)}{\partial x} \right|,$$

e quindi

$$\log p_X(x) = \log p_Z(f_\theta(x)) + \log \left| \det \frac{\partial f_\theta(x)}{\partial x} \right|.$$

La distribuzione p_Z è spesso una normale standard. L'addestramento massimizza la log-verosimiglianza dei campioni normali; il punteggio di anomalia in inferenza deve essere definito in modo coerente con la specifica implementazione. Una densità bassa non equivale direttamente a una probabilità calibrata di difetto.

Tra gli esempi della famiglia:

- **DifferNet** applica flussi alle feature per il rilevamento a livello di immagine;

- **CFLOW-AD** impiega flussi condizionali per la localizzazione;
- **FastFlow** utilizza trasformazioni bidimensionali sulle mappe di feature, mantenendo la struttura spaziale utile alla localizzazione [26].

FastFlow è il rappresentante selezionato nel progetto. La formulazione generale permette estrattori differenti, ma il confronto documentato utilizza un backbone convoluzionale congelato. Il Capitolo 3 distingue la funzione obiettivo del flusso dalla costruzione effettiva della mappa di anomalia.

2.3.5 Metodi basati sulla sintesi di anomalie

Un'ulteriore famiglia genera alterazioni artificiali dei campioni normali per costruire un segnale di apprendimento. L'assenza di esempi reali dei difetti non impedisce così l'impiego di etichette sintetiche per compiti di classificazione o segmentazione [7].

Per la segmentazione, una possibile componente dell'obiettivo è l'entropia incrociata binaria:

$$\ell(y, p) = -\big[y \log p + (1 - y) \log(1 - p)\big],$$

dove y indica se il pixel appartenga alla regione alterata e p è la previsione della rete. La funzione è calcolata sui pixel e aggregata secondo il metodo; possono essere presenti altre componenti di perdita.

Esempi di questa famiglia sono:

- **CutPaste**: ritaglia regioni di un'immagine normale, le modifica e le reinserisce per costruire un compito di riconoscimento delle alterazioni;
- **DRAEM**: combina una rete ricostruttiva con una rete discriminativa per apprendere la localizzazione di anomalie simulate [27];
- **NSA** (*Natural Synthetic Anomalies*): fonde patch provenienti da altre immagini attraverso tecniche di *Poisson image editing* [7].

La difficoltà principale riguarda la relazione tra difetti artificiali e reali: un modello può apprendere caratteristiche della procedura di sintesi che non siano rappresentative delle anomalie industriali. Questi metodi completano il quadro della letteratura, ma non sono inclusi nel confronto sperimentale.

2.3.6 Sintesi e collegamento con i modelli selezionati

La Tabella 2.2 riassume i principi e i limiti delle famiglie discusse. Le indicazioni sono qualitative: non costituiscono una graduatoria di accuratezza, memoria o velocità indipendente dall'implementazione.

I quattro modelli selezionati condividono l'adattamento sui soli campioni normali, ma differiscono nello spazio in cui descrivono la normalità e nel criterio con cui ne misurano le deviazioni. A partire da queste differenze, ho impostato il confronto considerando insieme discriminazione globale, localizzazione e comportamento delle mappe.

Tabella 2.2: Principi e principali limiti delle famiglie di metodi.

Famiglia	Principio ed esempi	Limiti da considerare
Descrittori manuali	Feature progettate e modelli statistici; SIFT, GMM, OC-SVM	Scelta delle feature e sensibilità alle condizioni di acquisizione
Ricostruzione	Discrepanza tra input e ricostruzione; autoencoder, CCW-SSIM	Possibile ricostruzione anche dei difetti
Rappresentazioni	Confronto con feature normali o statistiche; PaDiM, PatchCore	Dipendenza dall'estrattore e costo di memoria
Distillazione	Discrepanza Teacher–Student; RD4AD	Dipendenza dalle feature e dalla capacità dello Student
Flussi di normalizzazione	Trasformazione invertibile delle feature; FastFlow	Scelte dell'estrattore, dei flussi e del punteggio
Sintesi di anomalie	Apprendimento da alterazioni artificiali; CutPaste, DRAEM	Corrispondenza tra anomalie sintetiche e reali

2.4 Sfide aperte e limiti di applicazione

Le prestazioni su un benchmark non esauriscono i requisiti di un sistema industriale. Il trasferimento richiede di considerare acquisizione dei dati, cambiamenti del dominio e risorse disponibili [1, 15]. Le questioni seguenti costituiscono un quadro interpretativo, da distinguere dalle proprietà effettivamente misurate nel progetto.

2.4.1 Differenze di dominio e adattamento delle feature

I backbone pre-addestrati su dataset generici, come ImageNet, rendono disponibili rappresentazioni senza richiedere di apprendere da zero l'intero estrattore [20]. Tuttavia, le caratteristiche utili alla classificazione di immagini naturali possono non descrivere in modo ottimale microstrutture e difetti industriali.

L'adattamento delle feature può ridurre questa distanza, ma richiede di verificare che la rappresentazione conservi la capacità di distinguere anomalie non osservate [15]. Nel progetto, PaDiM, RD4AD e FastFlow condividono un backbone congelato; CCW-SSIM apprende invece dai soli campioni industriali. Questa differenza deve essere considerata nell'interpretazione dei risultati.

2.4.2 Scalabilità computazionale ed edge computing

L'esecuzione su dispositivi periferici può imporre vincoli di memoria, latenza ed energia, discussi sia per l'ispezione visiva industriale sia, più in generale, per le reti IoT [15, 16]. Il costo dipende dall'estrattore, dalla risoluzione, dai componenti addestrabili e dal trattamento delle mappe.

Nei metodi con memory bank, la selezione di un coreset limita la quantità di feature conservate, ma la consultazione dei riferimenti rimane parte dell'inferenza [20]. Nei modelli neurali, il costo dell'addestramento deve essere distinto dal tempo di predizione. Per questo, un tempo di addestramento basso non dimostra da solo l'idoneità a operare in tempo reale.

2.4.3 Cambiamenti del dominio e apprendimento continuo

Molti esperimenti assumono che le condizioni di addestramento e test siano stabili. Nelle applicazioni possono invece cambiare illuminazione, strumenti di acquisizione o caratteristiche del prodotto. Il modello appreso può così perdere validità e produrre più falsi allarmi.

La *continual anomaly detection* studia aggiornamenti incrementali della normalità, affrontando il rischio di dimenticare informazioni precedenti (*catastrophic forgetting*) o di incorporare anomalie non riconosciute. Una rassegna recente di questi temi è disponibile come preprint, non sottoposto a peer review [11]. La presenza di anomalie mai viste va distinta da questi cambiamenti della normalità: sono due aspetti diversi della generalizzazione.

2.4.4 Approcci zero-shot e foundation models

Quando è difficile raccogliere campioni normali specifici del prodotto, gli approcci *zero-shot* cercano di rilevare anomalie senza adattamento sul dominio di destinazione. Modelli visivo-linguistici e altri foundation models possono fornire rappresentazioni e indicazioni semantiche mediante prompt.

Il framework DCS esplora, ad esempio, l'integrazione di DINO, CLIP e SAM per il rilevamento e la segmentazione [24]. Queste possibilità ampliano il quadro della ricerca, ma richiedono valutazioni specifiche della precisione locale e dei costi. Non fanno parte dei metodi sperimentati nel presente lavoro.

La domanda da portare nella sperimentazione è quindi circoscritta: come si comportano quattro diverse pipeline one-class quando sono adattate e valutate sullo stesso caso di studio? Per affrontarla, nel capitolo seguente descrivo le configurazioni utilizzate e le condizioni del confronto.

Capitolo 3

Modelli selezionati e implementazione sperimentale

Il passaggio dallo stato dell'arte alla sperimentazione richiede di tradurre i diversi modi di apprendere la normalità in pipeline concrete. Per rendere leggibile il confronto, descrivo come vengono elaborate le immagini, quali componenti apprendono dai campioni normali e come viene costruita la mappa di anomalia. Questo percorso riguarda i quattro modelli selezionati: **CCW-SSIM**, **PaDiM**, **RD4AD** e **FastFlow**.

Il punto di partenza comune è costituito dalle immagini di **MVTec AD**, impiegate per l'adattamento al dominio industriale e per il test. Le rappresentazioni iniziali hanno invece una provenienza diversa: PaDiM, RD4AD e FastFlow utilizzano backbone con pesi pre-addestrati su **ImageNet**, mentre CCW-SSIM apprende da zero sui campioni normali del materiale. Questa differenza aiuta a comprendere sia le procedure di apprendimento sia i costi delle configurazioni adottate.

3.1 Criteri di selezione e ruolo dei modelli

Ho selezionato i modelli in modo da confrontare quattro famiglie metodologiche discusse nel Capitolo 2:

- **CCW-SSIM** [3] appartiene ai metodi basati sulla ricostruzione. Un autoencoder apprende a riprodurre i campioni normali e la discrepanza tra immagine originale e ricostruita è valutata considerando congiuntamente struttura e colore;

- **PaDiM** [8] è un metodo basato sulla rappresentazione. Le feature estratte da una rete pre-addestrata sono modellate, per ciascuna posizione spaziale, mediante una distribuzione gaussiana multivariata;

- **RD4AD** [10] adotta la *Reverse Distillation*. Un encoder *Teacher* congelato fornisce le rappresentazioni di riferimento, mentre un collo di bottiglia e un decoder *Student* apprendono a ricostruirle sui soli campioni normali;

- **FastFlow** [26] impiega flussi di normalizzazione bidimensionali. Trasformazioni invertibili apprendono la distribuzione delle feature normali preservandone l'organizzazione spaziale.

I quattro approcci rappresentano pertanto modalità differenti di costruire un modello della normalità: ricostruzione dell'immagine, stima statistica locale delle feature, ricostruzione distillata delle feature e stima esplicita della loro densità.

L'implementazione adotta lo scenario *one-class* definito nel Capitolo 1: la fase di adattamento utilizza esclusivamente immagini prive di difetti. Le immagini normali e anomale della partizione di test sono elaborate nella successiva fase di inferenza; le etichette e le maschere dei difetti sono utilizzate per la valutazione, secondo il protocollo illustrato nel capitolo dedicato ai risultati.

All'interno di questo scenario, l'apprendimento assume forme diverse. CCW-SSIM, RD4AD e FastFlow ottimizzano parametri neurali; PaDiM stima le statistiche gaussiane delle feature normali mantenendo congelato il backbone. Nel confronto considero quindi sia modelli con addestramento iterativo sia un approccio di adattamento statistico. Prima di entrare nelle singole implementazioni, è utile chiarire i dati e l'organizzazione sperimentale comuni.

3.2 Ambiente e protocollo implementativo comune

3.2.1 Dataset MVTec AD e categoria leather

MVTec AD [4] raccoglie immagini industriali ad alta risoluzione, organizzate in categorie di oggetti e texture. Il repository conserva le quindici categorie del dataset; le configurazioni e i manifesti verificabili per i quattro modelli fanno tuttavia riferimento alla categoria `leather`.

Per tale categoria, la partizione `train/good` contiene 245 immagini normali. La partizione di test contiene 124 immagini: 32 normali e 92 appartenenti alle tipologie di difetto `color`, `cut`, `fold`, `glue` e `poke`. Per le immagini difettose sono disponibili 92 maschere nella directory `ground_truth`. La medesima organizzazione dei dati è condivisa da tutte le implementazioni.

3.2.2 Organizzazione del repository e pipeline di esecuzione

Il repository comprende una directory per ciascun modello: `CCW-SSIM`, `PaDiM`, `RD4AD` e `FastFlow`. Ogni directory contiene un punto di ingresso `Main.py`, una configurazione dedicata e i moduli per caricamento dei dati, costruzione del

modello, adattamento e inferenza. Il notebook `pipeline_tesi.ipynb` coordina l'esecuzione.

Il dataset è conservato nella directory centrale `MVTec_Data` condivisa tra le singole directory locali dei modelli. L'esecuzione comune è articolata nelle seguenti fasi:

- adattamento del modello sui campioni normali presenti in `train/good`;
- caricamento del checkpoint o delle statistiche apprese;
- inferenza sulle immagini della partizione di test;
- salvataggio delle mappe di anomalia e dei metadati di esecuzione.

Il modulo `experiment_protocol.py` registra manifesti e impronte dei file impiegati, rendendo esplicite la configurazione e la provenienza degli artefatti.

3.2.3 Ambiente software e hardware

Gli ambienti registrati nei manifesti del progetto utilizzano Linux su architettura `x86_64` e Python 3.11.15. CCW-SSIM è implementato con TensorFlow 2.15 e le API Keras, mentre PaDiM, RD4AD e FastFlow utilizzano PyTorch 2.13.0 e Torchvision 0.28. Le configurazioni rilevate sono riassunte nella Tabella 3.1.

Modello	**Framework**	**Supporto GPU registrato**
CCW-SSIM	TensorFlow 2.15, API Keras	CUDA 12.2, cuDNN 8
PaDiM	PyTorch 2.13.0, Torchvision 0.28	CUDA 13.0
RD4AD	PyTorch 2.13.0, Torchvision 0.28	CUDA 13.0
FastFlow	PyTorch 2.13.0, Torchvision 0.28	CUDA 13.0

Tabella 3.1: Ambienti software registrati per le implementazioni.

La gestione dell'ambiente radice è definita dai file `pyproject.toml` e `uv.lock`. I manifesti indicano l'impiego di una GPU NVIDIA GeForce RTX 4070 Ti.

3.2.4 Gestione di configurazioni, checkpoint e artefatti

Tutte le pipeline fissano il seme pseudo-casuale a 1024. I parametri sono mantenuti nei rispettivi file di configurazione, mentre i pesi o le statistiche apprese sono salvati in formati specifici per il framework: file HDF5 per CCW-SSIM, file `pickle` per PaDiM e file PTH per RD4AD e FastFlow.

Le mappe di anomalia sono memorizzate come immagini TIFF in virgola mobile a 32 bit, senza quantizzazione preventiva. A ogni esecuzione sono inoltre associati un manifesto JSON e, per i modelli ottimizzati iterativamente, la cronologia di addestramento in formato CSV. Le mappe costituiscono il collegamento tra implementazione e valutazione: da esse saranno ricavati i punteggi delle immagini e le misure di localizzazione discusse nel Capitolo 4.

3.3 CCW-SSIM

3.3.1 Ricostruzione e confronto percettivo

CCW-SSIM è un metodo basato sulla ricostruzione [3]. Un autoencoder convoluzionale comprime l'immagine in una rappresentazione latente e la ricostruisce. L'addestramento sui soli campioni normali induce la rete a rappresentare regolarità cromatiche e strutturali proprie del dominio osservato.

Indicando con x l'immagine di ingresso e con g_θ l'autoencoder, la ricostruzione è

$$\widehat{x} = g_\theta(x). \tag{3.1}$$

Per valutare la ricostruzione, il modello combina una misura strutturale, derivata dalla CW-SSIM, con una componente cromatica percettiva. Il confronto richiede quindi una trasformazione dell'immagine che separi luminanza e crominanza.

La trasformazione S-CIELAB converte l'immagine RGB nello spazio XYZ, separa l'informazione in canali opponenti e applica filtri spaziali differenti alle componenti di luminanza e crominanza. Dopo la riconversione in CIELAB si ottengono i canali L^*, a^* e b^*. Il filtraggio tiene conto della diversa sensibilità del sistema visivo umano alle frequenze spaziali [3].

La componente strutturale è calcolata su coefficienti complessi ottenuti mediante una piramide steerable. Per due insiemi locali di coefficienti c_x e c_y, la CW-SSIM può essere espressa come

$$S_{\mathrm{CW}}(c_x, c_y) = \frac{2\left|\sum_i c_{x,i} c_{y,i}^*\right| + K}{\sum_i |c_{x,i}|^2 + \sum_i |c_{y,i}|^2 + K}, \tag{3.2}$$

dove $c_{y,i}^*$ è il complesso coniugato e K è una costante di stabilizzazione. La combinazione con la compatibilità cromatica consente di considerare sia le variazioni di struttura sia quelle di colore.

3.3.2 Architettura e configurazione adottata

L'architettura è definita in `NNModels.py`. Addestramento e inferenza sono gestiti rispettivamente da `Training.py` e `Prediction.py`; il preprocessing percettivo è implementato in `ColorUtils.py`, `SpatialFilters.py` e nella directory `Steerables`. La configurazione seleziona la variante seguente:

`Model_noise_skip_bigger_old`.

L'encoder utilizza convoluzioni 4×4 con canali 32, 64, 128, 256 e 2048 e stride rispettivamente 2, 2, 2, 4 e 4. Le attivazioni sono di tipo LeakyReLU. Il decoder è simmetrico e utilizza convoluzioni trasposte 4×4 con canali 256, 128, 64, 32 e 3 e stride 4, 4, 2, 2 e 2. L'ultimo strato ha attivazione lineare. Non sono presenti strati di Batch Normalization o Max-Pooling nella variante configurata.

Questa architettura apprende direttamente dalle immagini normali mediante la perdita `color_cwssim_loss`.[1] Il trattamento delle immagini e la formulazione della perdita determinano quindi quali regolarità strutturali e cromatiche guidano l'ottimizzazione.

3.3.3 Preprocessing e apprendimento della ricostruzione

Le immagini sorgente della categoria `leather` hanno dimensione 1024×1024 pixel. Il generatore estrae, da ciascuna immagine normale, 64 patch casuali di dimensione 128×128, che formano il batch effettivo. Sono configurate 200 epoche e 245 passi per epoca, corrispondenti al numero di immagini normali disponibili.

Il preprocessing converte RGB in XYZ mediante scikit-image, applica la trasformazione nei tre canali opponenti, filtra ciascun canale con kernel spaziali 5×5, riconverte in CIELAB e normalizza i valori dividendo per 102. L'ottimizzazione usa Adam con tasso di apprendimento iniziale 10^{-3}; uno scheduler moltiplica tale valore per 0,5 ogni 15 epoche. I tensori della pipeline CCW-SSIM sono elaborati in virgola mobile a 64 bit.

La perdita configurata combina la CW-SSIM sul primo canale Lab con l'errore cromatico sui canali a^* e b^*. La piramide complessa usa sei scale e sei orientazioni, mentre la finestra locale ha lato 7. I checkpoint sono salvati ogni dieci epoche.

La forma cromatica locale non coincide con la distanza euclidea descritta nella fonte. Indicando con e_{ab} l'errore quadratico medio dei due canali già divisi per 102, il codice di addestramento utilizza

$$e_{ab} = \frac{(a - \widehat{a})^2 + (b - \widehat{b})^2}{2}, \qquad C_{\text{train}} = 1 - \frac{\langle e_{ab} \rangle}{45}, \qquad \mathcal{L} = \langle 1 - S_{\text{CW}} C_{\text{train}} \rangle \,. \tag{3.3}$$

La media cromatica è calcolata sul batch e sulle coordinate spaziali. Nella fonte, invece, la componente è espressa tramite $\Delta E = \sqrt{(a^* - \widehat{a}^*)^2 + (b^* - \widehat{b}^*)^2}$ e un parametro k [3]. La pipeline locale mantiene la combinazione moltiplicativa di struttura e colore, ma costituisce una variante implementativa e non una traduzione letterale della metrica pubblicata.

3.3.4 Costruzione della mappa di anomalia

L'inferenza procede per porzioni dell'immagine (*patch*). Ogni immagine è suddivisa in patch 256×256 con stride di 16 pixel; le ricostruzioni sovrapposte sono ricomposte mediante media. La suddivisione del carico è controllata dal parametro `BatchSplits`, impostato a 25.

La mappa locale media la CW-SSIM alle altezze 7, 8 e 9 della piramide, su sei orientazioni, e applica alla componente cromatica una media spaziale 7×7.

[1] Il codice comprende anche componenti opzionali basati su VGG, esclusi dalla configurazione adottata.

Indicando tale media con $\mathcal{A}_7$, il punteggio salvato è

$$R = 1 - \overline{S}_{\mathrm{CW}} \left(1 - 10\mathcal{A}_7(e_{ab})\right). \tag{3.4}$$

Il coefficiente cromatico è quindi diverso da quello dell'addestramento: 10 anziché 1/45, un rapporto pari a 450. Differiscono anche la modalità di media e le altezze della piramide. Il codice non forza il punteggio nell'intervallo $[0, 1]$ prima del salvataggio.

La configurazione applica inoltre l'inversione del primo canale Lab ricostruito mediante `InvertFirstLabAxis=True`. La sua presenza è accertata, ma non dimostra una necessità del decoder; il test diagnostico discusso nel Capitolo 4 verifica separatamente questo punto. La pipeline registra checkpoint HDF5, cronologia CSV, manifesti JSON e mappe TIFF; la ricostruzione è calcolata in memoria per produrre la mappa. La funzione sperimentale per la predizione dell'intera immagine è presente nel codice, ma non è selezionata dalla configurazione utilizzata.

3.4 PaDiM

3.4.1 Rappresentazione multiscala e modello della normalità

PaDiM, acronimo di *Patch Distribution Modeling*, descrive ogni posizione dell'immagine mediante feature estratte a più livelli da una rete convoluzionale pre-addestrata [8]. Le rappresentazioni dei livelli iniziali mantengono maggiore dettaglio spaziale, mentre quelle più profonde codificano informazioni semantiche più astratte. L'allineamento spaziale e la concatenazione producono un vettore $z_{i,j}$ per ogni posizione (i, j).

Il backbone rimane congelato: ImageNet fornisce un'inizializzazione generica dell'estrattore, mentre la distribuzione normale specifica del dominio viene stimata sulle immagini MVTec AD.

Per ogni posizione spaziale, PaDiM stima una gaussiana multivariata

$$z_{i,j} \sim \mathcal{N}(\mu_{i,j}, \Sigma_{i,j}). \tag{3.5}$$

La discrepanza di una feature di test dalla distribuzione normale locale è espressa mediante la distanza di Mahalanobis

$$d_{i,j} = \sqrt{(z_{i,j} - \mu_{i,j})^{\mathrm{T}} \Sigma_{i,j}^{-1} (z_{i,j} - \mu_{i,j})}. \tag{3.6}$$

La modellazione indipendente delle posizioni conserva l'informazione relativa alla collocazione delle strutture nell'immagine.

3.4.2 Estrazione e allineamento delle feature

Il modello è definito in `NNModels.py`; adattamento e inferenza sono gestiti da `Training.py` e `Prediction.py`. Il backbone configurato è Wide ResNet-50-2 con pesi ImageNet-1K V1 forniti da Torchvision. I parametri del backbone restano congelati per l'intera procedura.

Le immagini sono ridimensionate a 224×224 e normalizzate con media e deviazione standard ImageNet. Le feature provengono da `layer1`, `layer2` e `layer3`, con 256, 512 e 1024 canali. Le mappe a risoluzione inferiore non sono semplicemente interpolate: la funzione `embedding_concat` usa operazioni `unfold` e `fold` per allineare e concatenare le feature in corrispondenza delle patch della mappa a risoluzione maggiore, pari a 56×56.

3.4.3 Stima delle distribuzioni locali

Le feature allineate costituiscono la rappresentazione sulla quale viene stimata la normalità. La concatenazione completa produce 1792 canali; per contenere la dimensionalità statistica, l'implementazione seleziona deterministicamente 550 indici di canale mediante `torch.randperm` con seme 1024. Non viene applicata una proiezione casuale: il sottoinsieme è composto direttamente dai canali selezionati e i relativi indici sono salvati insieme alle statistiche.

Le immagini normali sono elaborate in batch da 32, con due processi di caricamento e ordine non mescolato. La procedura esegue un unico passaggio sull'insieme di addestramento, calcola media e covarianza per ogni posizione e aggiunge $0{,}01I$ alla covarianza prima dell'inversione. Non sono presenti ottimizzatore o epoche di aggiornamento dei pesi. Le medie, le inverse delle covarianze e gli indici dei canali sono serializzati in un file `pickle`.

3.4.4 Costruzione della mappa di anomalia

In inferenza, il preprocessing e il sottoinsieme di canali coincidono con quelli usati durante l'adattamento. Per ogni posizione viene calcolata la distanza riportata nell'Equazione 3.6. La mappa 56×56 è riportata alla dimensione originale mediante interpolazione bicubica e regolarizzata con un filtro gaussiano di deviazione standard 4.

La pipeline salva le statistiche in formato `pickle`, il manifesto JSON e una mappa TIFF in virgola mobile a 32 bit per ciascuna immagine di test.

3.5 RD4AD

3.5.1 Ricostruzione delle feature mediante distillazione

Come PaDiM, RD4AD utilizza rappresentazioni pre-addestrate, ma apprende a ricostruirle attraverso una rete neurale. Il modello applica la distillazione della conoscenza in direzione inversa rispetto alla configurazione encoder–encoder tradizionale [10]. Il *Teacher Encoder*, pre-addestrato e congelato, produce rappresentazioni multiscala. Un decoder *Student* riceve una rappresentazione compressa e tenta di ricostruire le feature normali del Teacher procedendo dalle rappresentazioni profonde verso quelle meno profonde.

Durante l'adattamento, la somiglianza tra le rappresentazioni del Teacher e quelle ricostruite dallo Student guida l'ottimizzazione. In inferenza, la di-

screpanza locale tra le coppie di feature corrispondenti forma la mappa di anomalia.

Tra Teacher e Student è inserito un *One-Class Bottleneck Embedding* (OCBE). Il modulo aggrega le feature provenienti da livelli differenti e restringe il passaggio dell'informazione. L'asimmetria tra encoder congelato, collo di bottiglia addestrabile e decoder riduce la possibilità di trasferire direttamente allo Student tutte le attivazioni del Teacher.

Indicando con $f_k(x)$ la feature del Teacher al livello k e con $\widehat{f}_k(x)$ quella ricostruita, la discrepanza coseno locale impiegata per la localizzazione è

$$a_k(h,w) = 1 - \frac{f_k(h,w)^{\mathrm{T}} \widehat{f}_k(h,w)}{\|f_k(h,w)\|_2 \|\widehat{f}_k(h,w)\|_2}. \tag{3.7}$$

3.5.2 Configurazione di Teacher, OCBE e decoder

L'architettura è definita nel file `NNModels.py`. L'adattamento è gestito da `Training.py` e l'inferenza da `Prediction.py`.

Il Teacher è Wide ResNet-50-2 con pesi ImageNet-1K V1 ed espone le feature di `layer1`, `layer2` e `layer3`. Le immagini sono ridimensionate a 256×256 e normalizzate secondo i parametri ImageNet. I pesi del Teacher non vengono aggiornati.

Nel modulo OCBE, la prima feature viene ridotta spazialmente con due convoluzioni 3×3 a stride 2, ciascuna seguita da Batch Normalization e ReLU; la seconda è ridotta con un solo blocco analogo. Le due mappe sono concatenate con la terza feature e trasformate da un blocco residuo corrispondente al livello profondo della Wide ResNet. Il decoder utilizza blocchi residui con convoluzioni trasposte. Non sono presenti collegamenti *skip* diretti tra Teacher e Student; le connessioni residue appartengono ai blocchi interni del decoder.

La descrizione riguarda il percorso effettivamente utilizzato per ricostruire le feature.[2]

3.5.3 Apprendimento delle feature normali

L'addestramento usa batch da 16 immagini, due processi di caricamento e 200 epoche. L'ottimizzatore Adam aggiorna soltanto OCBE e Student Decoder con tasso di apprendimento 5×10^{-3} e coefficienti $\beta_1 = 0{,}5$, $\beta_2 = 0{,}999$.

Per ciascuno dei tre livelli, l'implementazione appiattisce tutte le dimensioni non appartenenti al batch, calcola la similarità coseno tra il vettore del Teacher e quello ricostruito e media sul batch. La perdita complessiva è la somma delle tre discrepanze:

$$\mathcal{L}_{\mathrm{RD}} = \sum_{k=1}^{3} \left(1 - \frac{1}{B} \sum_{b=1}^{B} \cos\Big(\mathrm{vec}(f_k^{(b)}), \mathrm{vec}(\widehat{f}_k^{(b)}) \Big) \right). \tag{3.8}$$

[2] La convoluzione 1×1 denominata `conv4` è definita nel codice, ma non viene richiamata dal metodo `forward` della configurazione corrente.

3.5.4 Costruzione della mappa di anomalia

In inferenza, la similarità coseno è invece calcolata lungo i canali per ogni coordinata spaziale, come nell'Equazione 3.7. Le tre mappe sono interpolate bilinearmente alla dimensione originale, sommate e filtrate con una gaussiana di deviazione standard 4.

Il checkpoint PTH contiene i parametri di OCBE e Student Decoder insieme ai metadati necessari; il backbone congelato è ricostruito separatamente dai pesi Torchvision. La pipeline produce inoltre cronologia CSV, manifesto JSON e mappe TIFF a 32 bit.

3.6 FastFlow

3.6.1 Modellazione delle feature con flussi invertibili

Anche FastFlow opera sulle feature di un estrattore pre-addestrato. La normalità viene descritta apprendendo trasformazioni invertibili: il modello rappresenta la distribuzione delle feature normali mediante flussi di normalizzazione bidimensionali [26]. In luogo di appiattire la rappresentazione, le trasformazioni operano su tensori con dimensioni di canale, altezza e larghezza, conservando la corrispondenza spaziale necessaria alla localizzazione.

Un backbone estrae feature a risoluzioni differenti. Per ciascuna scala, un flusso invertibile trasforma la feature x in una variabile latente z che segue una distribuzione normale standard. La densità nel dominio delle feature può essere calcolata esattamente mediante la formula del cambio di variabile.

Data una trasformazione invertibile $z = f_\theta(x)$, vale

$$\log p_X(x) = \log p_Z(f_\theta(x)) + \log \left| \det \frac{\partial f_\theta}{\partial x} \right| . \tag{3.9}$$

I blocchi di accoppiamento affine dividono i canali in due parti: una parte condiziona scala e traslazione applicate all'altra. Alternando suddivisioni e permutazioni, tutti i canali partecipano alle trasformazioni pur mantenendo calcolabili l'inversa e il determinante Jacobiano.

3.6.2 Estrazione e normalizzazione delle feature

L'implementazione è un porting locale derivato dal commit Anomalib `889a67f5`. Non richiede a runtime Lightning, Anomalib, timm o FrEIA: il backbone è fornito da Torchvision e le componenti necessarie dei flussi sono implementate nei moduli del progetto.

La configurazione supporta Wide ResNet-50-2 con pesi ImageNet-1K V1. Il backbone è congelato e produce le feature di `layer1`, `layer2` e `layer3`, aventi rispettivamente 256, 512 e 1024 canali e fattori di riduzione spaziale 4, 8 e 16. Le immagini sono ridimensionate a 256×256 e normalizzate con i parametri ImageNet. A ogni scala è applicata una Layer Normalization addestrabile sull'intero tensore (C, H, W).

La variante implementata non include backbone Transformer o Vision Transformer; tali possibilità appartengono alla formulazione generale del metodo, ma non alla configurazione del repository.

3.6.3 Configurazione e apprendimento dei flussi

Per ciascuna delle tre scale è costruito un `FastFlowBlock` composto da otto passi di accoppiamento invertibile. Poiché `Conv3x3Only` è disattivato, le sottoreti alternano kernel 3×3 e 1×1. Il rapporto dei canali nascosti è 1 e il parametro di *clamp* è 2.

Ogni passo esegue un accoppiamento affine, una trasformazione affine globale per canale e una permutazione deterministica inizializzata dal seme. La scala dell'accoppiamento è parametrizzata tramite una tangente iperbolica e un fattore 0,1; la scala globale usa una parametrizzazione *softplus*. Non è presente una fase separata di ActNorm dipendente dai dati.

L'addestramento usa batch da 8 immagini, due processi di caricamento e 200 epoche. Adam aggiorna Layer Normalization e flussi con tasso di apprendimento 10^{-3} e decadimento dei pesi 10^{-5}. Per ogni scala, la funzione obiettivo combina l'energia della variabile latente con il log-determinante Jacobiano:

$$\mathcal{L}_{\mathrm{FF}} = \sum_{k=1}^{3} \frac{1}{B} \sum_{b=1}^{B} \left(\frac{1}{2} \| z_k^{(b)} \|_2^2 - J_k^{(b)} \right), \tag{3.10}$$

dove $J_k^{(b)}$ è il log-determinante prodotto dal flusso alla scala k. I checkpoint intermedi sono salvati ogni cinque epoche e consentono la ripresa dell'esecuzione.

3.6.4 Costruzione della mappa di anomalia

La predizione utilizza batch da 4 immagini. Per ogni scala, il codice calcola la media sui canali di $-\frac{1}{2}z^2$, la trasforma esponenzialmente e ne cambia il segno per ottenere la mappa locale. Le mappe sono interpolate a 256×256 e aggregate mediante media; la pipeline le riporta poi alla dimensione originale e applica un filtro gaussiano con deviazione standard 4.

I checkpoint PTH contengono i parametri delle normalizzazioni e dei flussi, mentre il backbone congelato viene ricaricato separatamente. Sono prodotti checkpoint intermedi e finali, cronologia CSV, manifesto JSON e mappe TIFF in virgola mobile a 32 bit.

3.7 Sintesi delle configurazioni implementative

A questo punto è possibile mettere a confronto le procedure con cui i modelli apprendono dallo stesso insieme di immagini normali. La Tabella 3.2 raccoglie le configurazioni adottate, mettendo in relazione rappresentazione iniziale, dimensione dell'input e adattamento al caso di studio.

Modello	**Rappresentazione**	**Input e batch**	**Adattamento su MVTec AD**
CCW-SSIM	Autoencoder addestrato da zero	patch 128 × 128, batch 64	200 epoche, sole immagini normali
PaDiM	Wide ResNet-50-2 ImageNet congelata	224 × 224, batch 32	un passaggio per la stima statistica
RD4AD	Teacher ImageNet congelato, OCBE e decoder addestrabili	256 × 256, batch 16	200 epoche, sole immagini normali
FastFlow	Wide ResNet-50-2 ImageNet congelata e flussi addestrabili	256 × 256, batch 8	200 epoche, sole immagini normali

Tabella 3.2: Sintesi delle configurazioni implementative presenti nel repository.

Le differenze di risoluzione, rappresentazione e ottimizzazione sono parte delle condizioni del confronto. CCW-SSIM apprende la ricostruzione dell'immagine, PaDiM stima distribuzioni locali, RD4AD ricostruisce le feature del Teacher e FastFlow ne apprende trasformazioni invertibili. Tutte le pipeline producono infine mappe riferite alle immagini del medesimo insieme di test, sulle quali viene applicato il protocollo comune di valutazione.

Su queste mappe si basa l'analisi del capitolo seguente: quanto permettono di distinguere immagini normali e difettose e di localizzare le anomalie? Alla lettura delle metriche affianco gli esempi visivi, per collegare le prestazioni complessive alle risposte incomplete, diffuse o presenti nelle regioni normali. Nell'interpretare gli esiti tengo conto delle differenze implementative descritte, distinguendo le osservazioni dalle ipotesi sulle loro cause.

Capitolo 4

Analisi quantitativa e qualitativa dei risultati

Le mappe prodotte dalle pipeline descritte nel Capitolo 3 esprimono, per ogni immagine, la deviazione dalla normalità appresa. Per valutarne l'utilità nella categoria `leather` di MVTec AD, occorre leggere insieme il riconoscimento delle immagini difettose e la localizzazione delle anomalie. Le metriche descrivono il comportamento sull'intero insieme di test; le mappe e i casi di errore permettono di vedere come quel comportamento si manifesta nelle immagini.

Nella lettura dei risultati considero quindi la discriminazione delle immagini, la qualità della localizzazione e i costi osservati nella fase di apprendimento. Per confrontare gli esiti con le pubblicazioni di riferimento, tengo conto anche delle differenze tra configurazioni e protocolli di valutazione.

4.1 Perimetro e criteri di lettura

Il confronto utilizza l'insieme di addestramento di 245 immagini normali e l'insieme di test di 124 immagini descritti nel capitolo precedente. Nel test, 32 immagini sono normali e 92 presentano anomalie. La Tabella 4.1 ne riporta la distribuzione nelle cinque tipologie di difetto del materiale, fornendo il riferimento per la lettura degli errori.

I valori riportati provengono dai file di risultato archiviati nel repository e dal relativo rapporto di confronto. I manifesti registrano la configurazione, i dati, i checkpoint, le mappe e il valutatore impiegati. Per questa analisi non sono stati eseguiti nuovi addestramenti: le figure sono state generate dalle mappe archiviate e le matrici di confusione sono state ricostruite dagli stessi punteggi. La verifica di provenienza ha confermato la corrispondenza degli artefatti con gli hash registrati.[1]

[1] Le esecuzioni originarie sono state effettuate su Linux. La verifica riguarda l'integrità degli artefatti e l'analisi dei risultati archiviati, non una nuova esecuzione dei modelli.

Tipo di immagine	Numero
`good`	32
`color`	19
`cut`	19
`fold`	17
`glue`	19
`poke`	18
Totale	124

Tabella 4.1: Composizione dell'insieme di test della categoria `leather`.

Come definito nel Capitolo 1, l'AUROC a livello di immagine misura la capacità di ordinare le immagini normali e anomale. L'AUROC a livello di pixel valuta l'ordinamento dei pixel, mentre l'AU-PRO valuta la copertura delle regioni annotate al variare dei falsi positivi. Nel protocollo adottato, quest'ultima area è normalizzata nell'intervallo di FPR compreso tra 0 e 0,3. Le due misure di localizzazione sono complementari e non intercambiabili.

Le metriche F1 aggiungono una lettura a soglia. Il massimo F1 è ottenuto selezionando la soglia sull'insieme di test: è quindi una misura *oracle*, utile a descrivere il potenziale del punteggio, non una prestazione operativa su dati indipendenti. A livello di pixel il valutatore utilizza un campionamento con passo quattro lungo entrambe le dimensioni e una griglia di 500 soglie; i valori di precisione e richiamo discussi nel seguito si riferiscono a questo stesso campionamento.

Il protocollo comune rende confrontabili gli esiti delle pipeline, ma non annulla le differenze tra esse. PaDiM, RD4AD e FastFlow impiegano feature di una Wide ResNet-50-2 pre-addestrata su ImageNet; CCW-SSIM apprende invece la ricostruzione dalle immagini normali. Risoluzioni, modalità di addestramento e post-elaborazione rimangono quelli documentati nel Capitolo 3. Il confronto riguarda dunque sistemi completi nelle configurazioni adottate, non l'effetto isolato della sola funzione di perdita o della sola famiglia metodologica.

4.2 Confronto quantitativo complessivo

4.2.1 Discriminazione delle immagini e localizzazione

La Tabella 4.2 riassume le metriche principali. Tutti i valori sono espressi nell'intervallo $[0, 1]$ e arrotondati a quattro cifre decimali.

PaDiM, RD4AD e FastFlow raggiungono un'AUROC a livello di immagine pari a 1: i loro punteggi separano completamente le immagini normali da quelle difettose del test. La metrica raggiunge quindi il valore massimo per tutti e tre i metodi e non permette di distinguerli rispetto a questo compito. Il risultato si riferisce ai 124 campioni esaminati e a un'unica esecuzione per pipeline, con seme pseudo-casuale pari a 1024; la generalizzazione a nuove immagini richiede una valutazione ulteriore.

Metrica	CCW-SSIM	PaDiM	RD4AD	FastFlow
AUROC immagini	0,8991	1,0000	1,0000	1,0000
AUROC pixel	0,9707	0,9911	0,9942	0,9968
AU-PRO	0,8864	0,9786	0,9833	0,9928

Tabella 4.2: Risultati complessivi sull'insieme di test `leather`, con AU-PRO normalizzata fino a FPR pari a 0,3.

Per distinguere le pipeline occorre passare alla localizzazione. FastFlow presenta il valore più alto sia di AUROC pixel sia di AU-PRO, seguito da RD4AD e PaDiM. Rispetto a RD4AD, il vantaggio osservato di FastFlow è di circa 0,26 punti percentuali nell'AUROC pixel e di 0,95 punti percentuali nell'AU-PRO. Tra RD4AD e PaDiM le differenze sono rispettivamente di circa 0,30 e 0,46 punti percentuali. Leggo questi scarti come differenze descrittive: senza ripetizioni e intervalli di incertezza, non è possibile affermarne la significatività statistica.

Per CCW-SSIM l'AUROC pixel è pari a 0,9707 e l'AU-PRO a 0,8864. La prima misura indica un buon ordinamento complessivo dei pixel, mentre la seconda evidenzia una copertura delle regioni meno efficace nell'intervallo di falsi positivi considerato. Poiché le due curve integrano grandezze diverse, i valori vanno letti attraverso il loro significato: l'analisi delle mappe permetterà di osservare la copertura dei difetti e le risposte sulla texture normale.

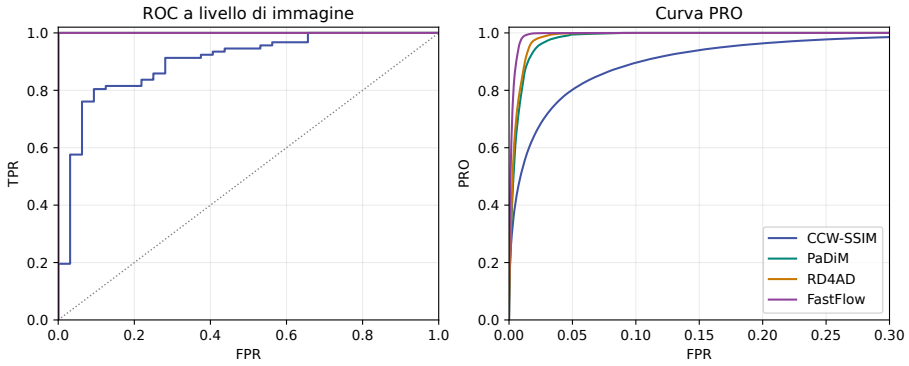


Figura 4.1: Curve ROC a livello di immagine e curve PRO ricavate dai punti archiviati dal valutatore. Le curve ROC di PaDiM, RD4AD e FastFlow si sovrappongono. Le aree riportate nel testo sono quelle salvate nei risultati, non una nuova integrazione dei punti visualizzati.

4.2.2 Risultati a soglia e casi di errore

Le aree sotto le curve descrivono il comportamento al variare della soglia. Per esaminare le decisioni sulle singole immagini e la segmentazione dei pixel, la

Tabella 4.3 affianca a queste misure i risultati nei punti operativi selezionati dal valutatore.

Metrica	CCW-SSIM	PaDiM	RD4AD	FastFlow
F1 massimo immagini	0,9081	1,0000	1,0000	1,0000
F1 massimo pixel	0,3840	0,4243	0,4981	0,6161
FPR con TPR $\geq 95\%$	0,5312	0,0000	0,0000	0,0000

Tabella 4.3: Metriche a soglia. Le soglie di massimo F1 sono selezionate sull'insieme di test; l'FPR dell'ultima riga riguarda la classificazione delle immagini.

Alla soglia che massimizza l'F1 a livello di immagine, CCW-SSIM identifica 84 immagini difettose e ne omette 8; classifica correttamente 23 immagini normali e segnala come anomale le altre 9. La precisione è circa 0,9032 e il richiamo circa 0,9130. Per gli altri tre metodi, alla rispettiva soglia ottimale, non si osservano falsi positivi o falsi negativi a livello di immagine.

Il comportamento cambia imponendo un richiamo di almeno il 95%: per CCW-SSIM l'FPR sale a 0,5312, corrispondente a 17 immagini normali segnalate su 32. Il valore illustra il costo, su questo insieme di dati, della riduzione delle omissioni. Non va confuso con i nove falsi positivi alla soglia di massimo F1, poiché i due criteri selezionano punti operativi differenti.

A livello di pixel, i massimi F1 sono sensibilmente inferiori alle AUROC pixel. Un buon ordinamento dei punteggi non implica infatti che esista una soglia capace di delineare il difetto con precisione elevata: possono rimanere aloni, risposte sullo sfondo o parti della regione non rilevate. La Tabella 4.4 esplicita questo compromesso.

Modello	Precisione pixel	Richiamo pixel	F1 pixel
CCW-SSIM	0,3704	0,3986	0,3840
PaDiM	0,3159	0,6463	0,4243
RD4AD	0,3999	0,6603	0,4981
FastFlow	0,5233	0,7488	0,6161

Tabella 4.4: Precisione e richiamo alla soglia globale di massimo F1 pixel di ciascun modello, sul campionamento previsto dal valutatore.

PaDiM recupera una quota di pixel anomali maggiore di CCW-SSIM, ma presenta una precisione inferiore. RD4AD migliora entrambi i valori rispetto a PaDiM, mentre FastFlow ottiene il miglior compromesso osservato. Anche per FastFlow una precisione di 0,5233 indica che, alla soglia considerata, una parte consistente dei pixel segnalati non appartiene alle annotazioni del difetto. L'esito favorevole nel riconoscimento dell'immagine non deve quindi essere trasferito automaticamente alla precisione della segmentazione.

L'interpretazione di questi punti operativi dipende dalla scala del punteggio di ciascun modello. Il valutatore normalizza punteggi originariamente differenti, che non rappresentano probabilità calibrate: le soglie numeriche vanno quindi riferite alla rispettiva pipeline. Il loro uso su dati futuri richiederebbe

una selezione su un insieme di validazione indipendente, come discusso nella Sezione 4.5.

4.2.3 Costi osservati nella fase di apprendimento

Accanto alle prestazioni, i tempi registrati descrivono il lavoro necessario ad adattare ciascuna pipeline al materiale. La Tabella 4.5 li mette in relazione con i passaggi sui dati e gli aggiornamenti dei parametri, distinguendo la stima statistica di PaDiM dall'ottimizzazione iterativa degli altri modelli.

Pipeline	Passaggi sui dati	Aggiornamenti	Tempo (min)
CCW-SSIM	200	49 000	374,1
PaDiM	1	0	0,2
RD4AD	200	3 200	8,7
FastFlow	200	6 200	11,5

Tabella 4.5: Tempi della fase di apprendimento o adattamento registrati nelle esecuzioni su GPU NVIDIA RTX 4070 Ti. Per PaDiM l'adattamento consiste nella stima statistica, senza aggiornamenti tramite discesa del gradiente.

PaDiM richiede il tempo di adattamento più basso, mentre CCW-SSIM presenta quello più alto. RD4AD e FastFlow si collocano tra questi estremi. I tempi non rappresentano un confronto a parità di lavoro: differiscono il numero di aggiornamenti, il batch, la precisione numerica, la gestione delle patch e l'architettura. Inoltre, i metodi basati su feature utilizzano un pre-addestramento esterno, il cui costo non è incluso nelle misure.

Da questi tempi si può valutare il costo locale di preparazione delle pipeline. Per discuterne l'efficienza complessiva servirebbero anche misure omogenee di latenza, numero di immagini elaborate per unità di tempo e memoria durante l'inferenza, che qui non sono disponibili. Il tempo di addestramento non può sostituirle né verificare le dichiarazioni di velocità dei lavori originali.

4.3 Analisi qualitativa e confronto caso per caso

Per comprendere le differenze tra AUROC, AU-PRO e F1, ho affiancato ai valori numerici l'esame della distribuzione dei punteggi nelle immagini. Le figure mostrano l'immagine originale, la maschera annotata e le quattro mappe di anomalia. Il contorno verde sulle mappe indica il riferimento annotato, non una segmentazione prodotta dal modello. Per mantenere una scala coerente tra esempi dello stesso metodo, la visualizzazione applica una normalizzazione min–max basata sull'intero insieme delle mappe di test di ciascuna pipeline:

$$\widetilde{S}_m(x) = \frac{S_m(x) - a_m}{b_m - a_m}, \tag{4.1}$$

dove a_m e b_m sono rispettivamente il minimo e il massimo delle mappe del modello m. Non viene normalizzata ogni immagine separatamente. Questa trasformazione serve soltanto alla visualizzazione e non modifica le metriche archiviate.

Non coincide necessariamente con gli estremi ricavati dal campionamento usato per l'F1 pixel.

Poiché ciascun modello mantiene una propria scala, la stessa tonalità in due pannelli non esprime una probabilità di anomalia equivalente. Il confronto visivo riguarda posizione, forma ed estensione delle risposte. La classificazione dipende invece dal massimo della mappa e dalla soglia globale del modello: una risposta su parte del difetto può essere sufficiente a segnalare l'immagine, mentre una zona luminosa in un campione normale va interpretata rispetto a quella soglia. I conteggi per tipologia riportati nel seguito descrivono le decisioni sulle immagini; la qualità della segmentazione è valutata dalle metriche a livello di pixel e regionali.

Per ciascuna tipologia di difetto ho scelto il primo file in ordine lessicografico, `000.png`. Questa regola rende la selezione riproducibile, pur senza garantire che i campioni siano rappresentativi dell'intera tipologia. Completano la lettura due esempi diagnostici: l'immagine normale con il massimo punteggio CCW-SSIM e un difetto omesso da tale modello. Prima di esaminare i singoli casi, la Tabella 4.6 riporta le omissioni sull'intero insieme di test.

Difetto	CCW-SSIM	PaDiM	RD4AD	FastFlow
`color`	2/19	0/19	0/19	0/19
`cut`	1/19	0/19	0/19	0/19
`fold`	0/17	0/17	0/17	0/17
`glue`	4/19	0/19	0/19	0/19
`poke`	1/18	0/18	0/18	0/18
Totale	8/92	0/92	0/92	0/92

Tabella 4.6: Falsi negativi a livello di immagine sul numero di campioni difettosi. Per ogni modello si mantiene la stessa soglia globale di massimo F1 immagini, senza ottimizzarla per tipologia.

4.3.1 Alterazione cromatica: `color`

Nel campione `color/000.png` il difetto occupa una piccola zona nella parte inferiore destra dell'immagine. PaDiM, RD4AD e FastFlow producono una risposta riconoscibile nella posizione annotata. PaDiM presenta una distribuzione più regolare e a blocchi, RD4AD una risposta più compatta, mentre FastFlow evidenzia il difetto con un nucleo intenso e una risposta anche nelle aree circostanti. La mappa CCW-SSIM è maggiormente influenzata dalla tessitura diffusa del materiale e mostra un contrasto locale meno evidente.

Il contrasto locale poco evidente di CCW-SSIM si accompagna al riconoscimento di 17 delle 19 immagini `color` alla soglia globale di massimo F1. Il punto da approfondire è quindi la relazione tra risposta cromatica e tessitura del materiale. Per valutare il contributo specifico del colore alla funzione di perdita occorre però un'ablazione controllata rispetto alla variante priva di tale componente; questo punto verrà ripreso nel confronto con la fonte.

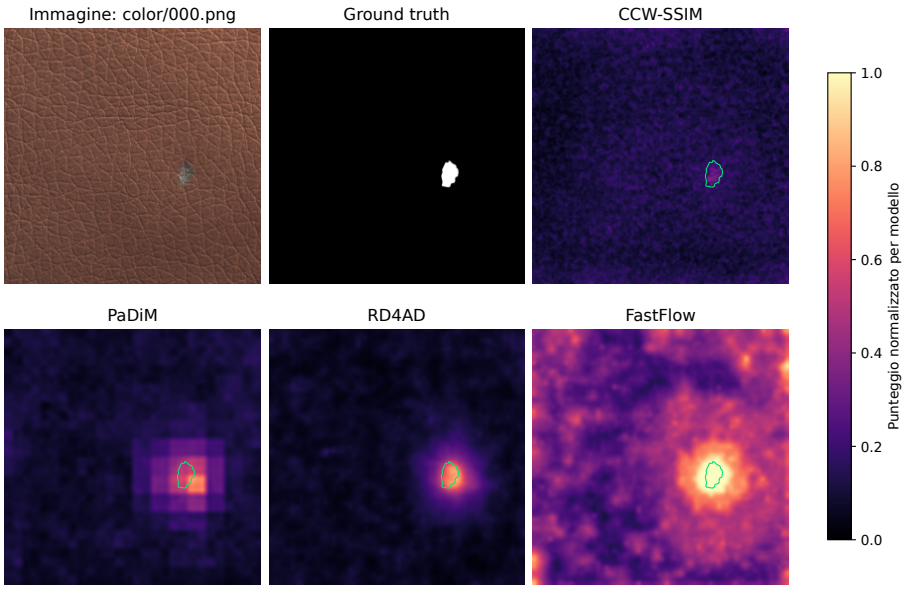


Figura 4.2: Confronto qualitativo sul campione `color/000.png`. Il contorno verde rappresenta la regione annotata.

4.3.2 Taglio: `cut`

Il campione `cut/000.png` contiene un difetto sottile, prevalentemente orizzontale. I tre metodi basati su feature rispondono nell'area corretta, ma la regione ad alto punteggio si estende oltre la geometria del taglio. Questo esempio rende visibile la distinzione tra individuare la posizione del difetto e ricostruirne precisamente il contorno. La risposta PaDiM è più squadrata; quelle di RD4AD e FastFlow sono più continue, pur mantenendo un'estensione superiore alla maschera.

CCW-SSIM presenta una mappa granulare nella quale il difetto emerge meno nettamente rispetto allo sfondo; sull'intera tipologia viene omessa una sola immagine su 19. Nel leggere questo caso, è utile concentrarsi sui bordi: per una struttura sottile, ricampionamento e filtraggio possono influenzare l'estensione visibile della risposta. Resta da verificare quanto ciascun passaggio contribuisca all'esito, perché gli esperimenti disponibili riguardano la pipeline completa.

4.3.3 Piega: `fold`

In `fold/000.png` la regione difettosa è allungata e si trova nella parte superiore sinistra. PaDiM, RD4AD e FastFlow individuano tale zona, con risposte che ne comprendono anche il contesto. CCW-SSIM conserva una risposta diffusa alla texture e un'evidenza locale meno netta alla scala di visualizzazione adottata.

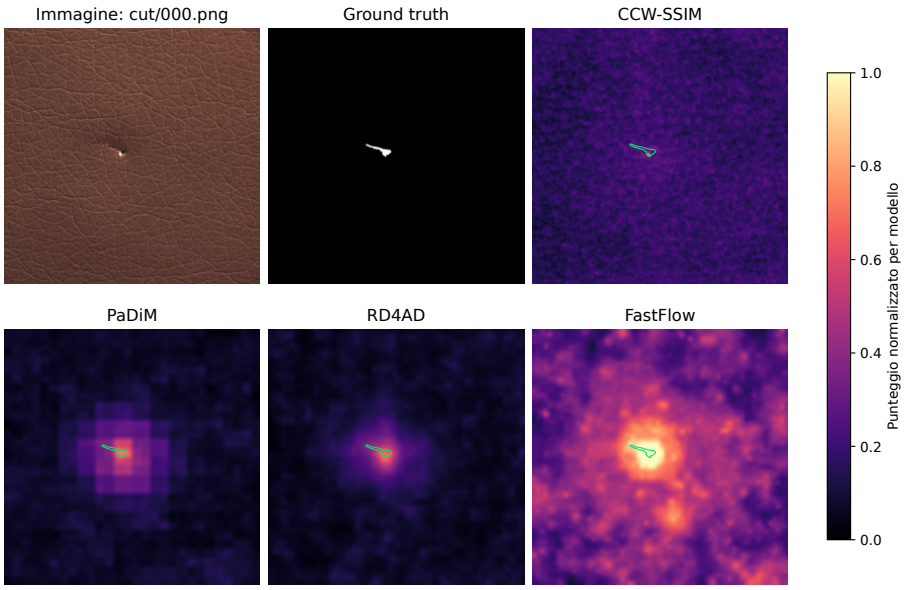


Figura 4.3: Confronto sul campione `cut/000.png`: la risposta nell'area corretta non implica una segmentazione aderente al taglio.

CCW-SSIM riconosce tutte le 17 immagini `fold` alla soglia di massimo F1 immagini. Questo risultato è utile per interpretare la risposta diffusa osservata nel pannello: un contrasto visivo limitato della regione difettosa può convivere con il riconoscimento completo della tipologia nel test. Il massimo della mappa può infatti essere sufficiente a sostenere la decisione sull'immagine.

4.3.4 Residuo di colla: `glue`

Nel campione `glue/000.png` la regione annotata è compatta e presenta variazioni di aspetto al suo interno. Le mappe PaDiM e RD4AD rispondono in modo non uniforme nella zona difettosa; FastFlow produce un nucleo più estesamente intenso. CCW-SSIM mostra risposte soprattutto su piccole porzioni e variazioni locali, senza riprodurre con altrettanta evidenza l'intera area annotata.

Quattro degli otto falsi negativi di CCW-SSIM appartengono a `glue`: sono quattro omissioni su 19 campioni, il numero più alto tra le tipologie considerate. La Figura 4.6 aggiunge `glue/001.png`, il primo falso negativo in ordine lessicografico di questa tipologia. Il campione è selezionato per documentare un errore effettivo, non per rappresentare un caso medio. La sua omissione è determinata dal punteggio globale dell'immagine alla soglia considerata, non dall'assenza di qualunque risposta visibile sulla mappa.

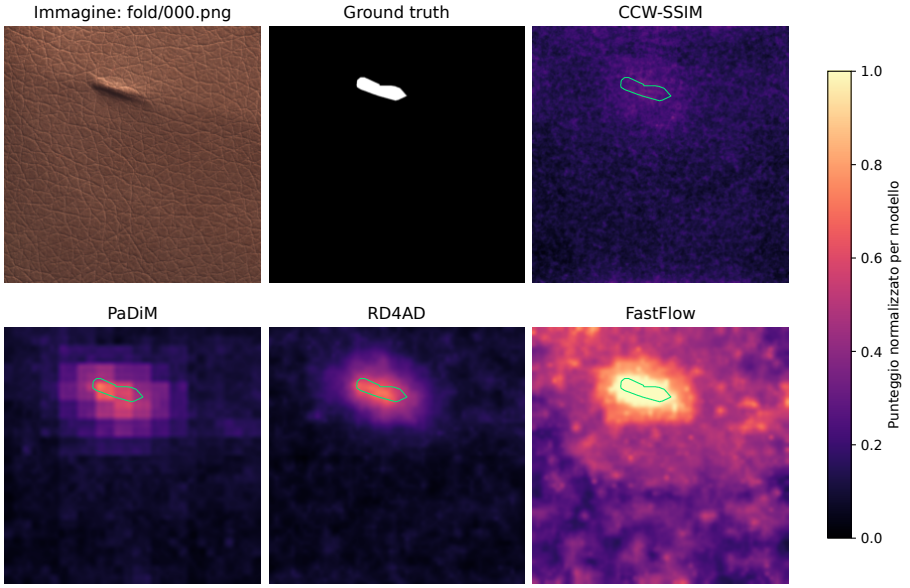


Figura 4.4: Confronto sul campione `fold/000.png`, caratterizzato da un difetto allungato.

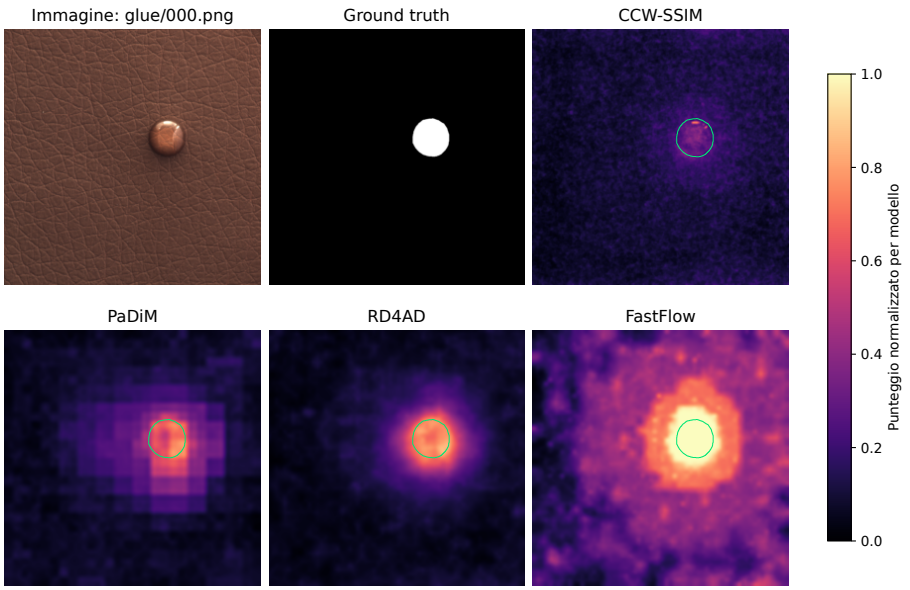


Figura 4.5: Confronto sul campione `glue/000.png`.

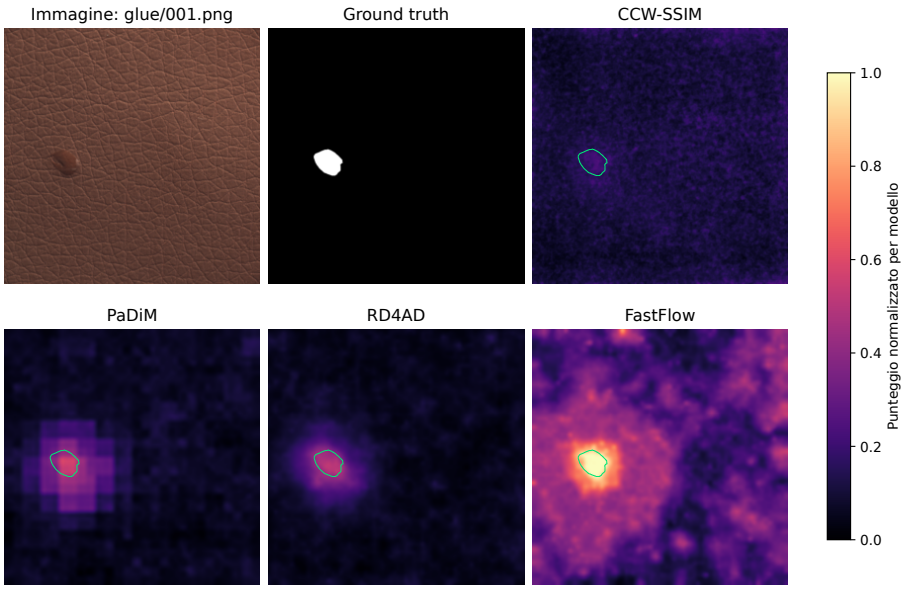


Figura 4.6: Campione `glue/001.png`, falso negativo di CCW-SSIM alla soglia globale di massimo F1 immagini. Gli altri tre modelli lo classificano correttamente alle rispettive soglie.

4.3.5 Perforazione: `poke`

In `poke/000.png` il difetto è piccolo e si trova nella parte inferiore sinistra. I tre modelli basati su feature presentano una risposta localizzata, più ampia del riferimento annotato. CCW-SSIM mostra una risposta meno distinguibile dalla texture circostante. Per un difetto di dimensioni ridotte, un'estensione della mappa oltre il contorno può incidere sulla precisione anche quando il centro della risposta è corretto.

CCW-SSIM omette una delle 18 immagini `poke` alla soglia globale scelta, mentre gli altri metodi le riconoscono tutte. Rispetto agli esempi precedenti, la piccola area annotata mette in evidenza il peso delle risposte circostanti: una localizzazione centrata sul difetto può comunque includere una quota rilevante di pixel normali.

4.3.6 Immagine normale e falso positivo

Il campione `good/030.png` è l'immagine normale con il massimo punteggio CCW-SSIM. La maschera di riferimento è vuota. La mappa CCW-SSIM contiene risposte diffuse alla tessitura e picchi locali: il massimo supera la soglia di classificazione e rende questa immagine uno dei nove falsi positivi del modello.

Anche le altre pipeline producono mappe non uniformi su questa immagine normale, ma i rispettivi punteggi rimangono al di sotto delle soglie globali di

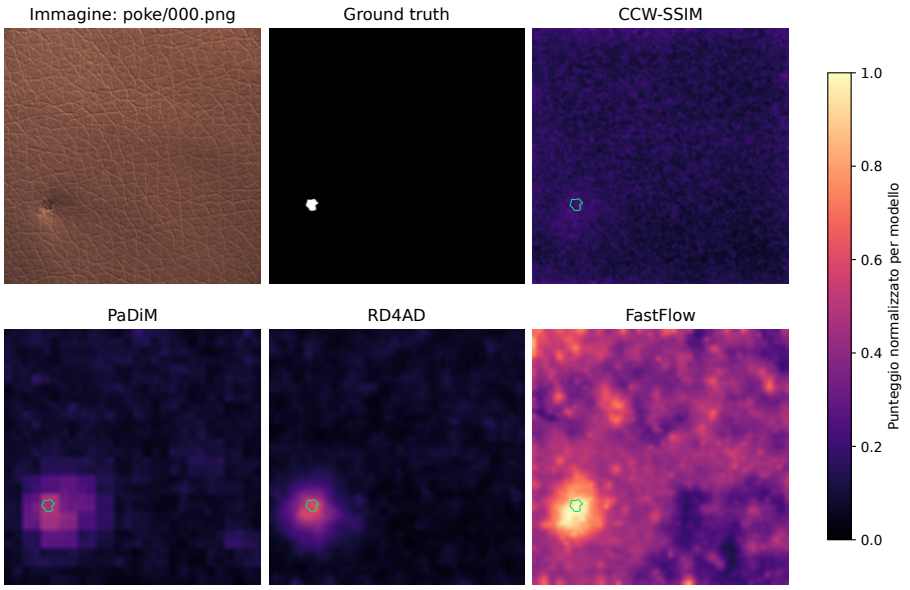


Figura 4.7: Confronto sul campione `poke/000.png`, con difetto di piccole dimensioni.

massimo F1 immagini. Per CCW-SSIM, invece, il picco determina una segnalazione errata. Da questo caso emerge una domanda per un successivo approfondimento: aggregazioni alternative al massimo potrebbero ridurre le segnalazioni legate alla risposta sulla tessitura? Il loro beneficio rimane da verificare.

4.3.7 Sintesi della lettura qualitativa

Negli esempi selezionati, PaDiM mostra risposte più regolari e a blocchi, RD4AD risposte più compatte e FastFlow nuclei intensi accompagnati, in alcuni casi, da un'attivazione più diffusa del contesto. CCW-SSIM presenta mappe più granulari, nelle quali la texture normale rimane visibile e il contrasto del difetto può essere ridotto. Queste descrizioni riguardano i pannelli esaminati e non sostituiscono un'analisi quantitativa di tutte le mappe.

La lettura delle figure è coerente con due risultati complessivi: i metodi basati su feature individuano la zona difettosa nei campioni mostrati, ma mantengono imprecisioni di estensione; CCW-SSIM può segnalare un difetto senza coprirlo interamente e può attribuire un punteggio elevato anche a un'immagine normale. Non è invece giustificato attribuire le differenze alla sola risoluzione, al solo pre-addestramento o alla sola funzione di perdita: tali fattori non sono stati variati separatamente.

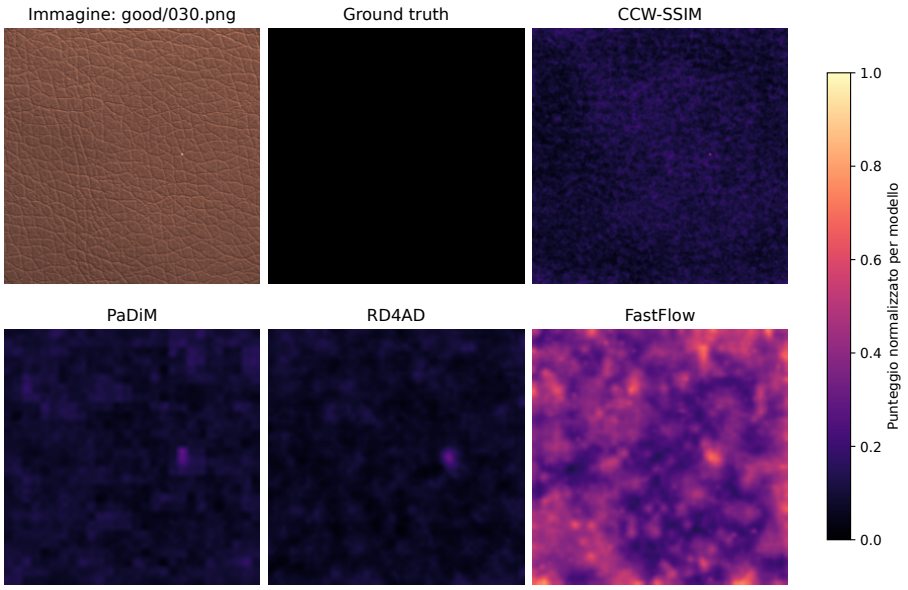


Figura 4.8: Immagine normale `good/030.png`, falso positivo di CCW-SSIM alla soglia di massimo F1 immagini. L'assenza di regioni annotate è mostrata dalla maschera vuota.

4.4 Confronto con le pubblicazioni di riferimento

Dopo aver esaminato metriche e mappe, il passo successivo è mettere i risultati in relazione con le proposte originali. Nel confronto con le fonti considero sia i valori pubblicati sia le condizioni in cui sono stati ottenuti. Il riferimento è costituito dai risultati di `leather`, quando disponibili, mantenuti distinti dalle medie sulle quindici categorie di MVTec AD.

La Tabella 4.7 affianca i risultati locali ai valori pubblicati per le metriche con una corrispondenza nominale. Gli scarti sono espressi in punti percentuali e sono soltanto descrittivi: la tabella non presuppone identità dei protocolli. CCW-SSIM viene discusso separatamente, poiché la diversa definizione dell'area impedisce di riportare uno scarto equivalente.

4.4.1 PaDiM

Nella Tabella III del lavoro di Defard et al. [8], la configurazione PaDiM con Wide ResNet-50-2 e 550 dimensioni riporta per `leather` un'AUROC pixel del 99,2% e un PRO-score del 97,8%. I valori locali, rispettivamente 99,11% e 97,86%, sono prossimi a quelli pubblicati. Questo riscontro sostiene la coerenza del comportamento di localizzazione osservato.

Per interpretare questo allineamento occorre considerare il preprocessing. Il lavoro originale prevede ridimensionamento a 256 pixel e ritaglio centrale a

Metodo	Metrica	Fonte (%)	Locale (%)	Scarto (p.p.)
PaDiM	AUROC pixel	99,20	99,11	$-0,09$
	PRO	97,80	97,86	$+0,06$
RD4AD	AUROC immagini	100,00	100,00	$0,00$
	AUROC pixel	99,40	99,42	$+0,02$
	PRO	99,10	98,33	$-0,77$
FastFlow	AUROC immagini	100,00	100,00	$0,00$
	AUROC pixel	99,50	99,68	$+0,18$

Tabella 4.7: Riscontro descrittivo su `leather` con le tabelle originali di PaDiM, Reverse Distillation e FastFlow. La misura regionale locale è l'AU-PRO definita nel protocollo dell'elaborato; la corrispondenza esatta dei valutatori non è accertata per tutte le fonti. Lo scarto è locale meno fonte e non indica significatività statistica.

224, mentre la configurazione locale ridimensiona direttamente a 224. Possono inoltre differire dettagli del valutatore e della selezione delle feature.

La differenza di preprocessing è concreta: il ritaglio conserva la scala della parte centrale dell'immagine ridimensionata e ne esclude i margini, mentre il ridimensionamento diretto mantiene l'intero campo visivo a una scala diversa. Questo modifica la corrispondenza tra texture, difetti e posizioni delle feature statistiche. È un fattore plausibile per leggere i piccoli scarti, ma il suo effetto non è stato misurato separatamente. Anche il rapido adattamento locale è coerente con l'assenza di addestramento della rete descritta dalla pubblicazione; non costituisce invece una verifica dei suoi tempi assoluti, ottenuti con hardware e implementazione differenti.

4.4.2 Reverse Distillation

Le Tabelle 1 e 2 di Deng e Li [10] riportano per `leather`, alla risoluzione di 256 pixel, un'AUROC immagini del 100%, un'AUROC pixel del 99,4% e un PRO del 99,1%. Nell'esecuzione locale RD4AD raggiunge rispettivamente il 100%, il 99,42% e il 98,33%.

La discriminazione delle immagini e l'AUROC pixel risultano quindi prossime ai valori pubblicati, mentre tra i due valori regionali vi è uno scarto descrittivo di circa 0,77 punti percentuali. La verifica del codice ufficiale [9] accerta però protocolli differenti: il riferimento calcola il PRO per ciascuna immagine difettosa, con 200 soglie, poi media le aree; il valutatore locale costruisce un'unica curva globale sulle 124 immagini, includendo quelle normali. Le due procedure pesano diversamente regioni e falsi positivi. Lo scarto non misura quindi una perdita del modello a parità di valutazione.

I principali parametri di addestramento coincidono con quelli dichiarati nella fonte: 200 epoche, batch da 16 immagini e Adam con tasso di apprendimento pari a 0,005 e coefficienti $(0{,}5, 0{,}999)$. Il codice ufficiale valuta e filtra a 256×256, mentre qui le mappe sono interpolate a 1024×1024 prima del filtro con $\sigma = 4$: lo stesso valore numerico non corrisponde alla stessa scala spaziale. È verificata

la differenza di procedura, non il suo contributo quantitativo al risultato. Per isolare le cause servirebbe applicare entrambi i protocolli a predizioni omogenee, senza modificare retrospettivamente il confronto locale.

4.4.3 FastFlow

La Tabella 2 di Yu et al. [26] riporta per `leather` un'AUROC immagini del 100% e un'AUROC pixel del 99,5%. L'esecuzione locale ottiene il 100% e il 99,68%. I risultati confermano, nel perimetro considerato, la buona separazione delle immagini e l'elevata capacità di localizzazione descritta nella pubblicazione.

Il porting adottato utilizza la configurazione Wide ResNet-50-2 documentata nel Capitolo 3, mentre la pubblicazione considera diverse architetture di estrazione delle feature. L'identità completa con la configurazione sottostante al valore pubblicato non è stabilita. Il lieve scarto positivo va quindi interpretato insieme alle scelte di implementazione e di apprendimento.

La fonte descrive 500 epoche e batch da 32, mentre il progetto usa 200 epoche e batch da 8, mantenendo tasso di apprendimento 10^{-3} e decadimento dei pesi 10^{-5}. Cambiano così il regime di ottimizzazione e il numero di aggiornamenti. Il vantaggio descrittivo di 0,18 punti percentuali non può essere attribuito a una modifica specifica senza prove controllate. Il confronto sostiene la capacità di produrre punteggi discriminanti nel caso di studio; le proprietà di efficienza in inferenza discusse dalla pubblicazione richiederebbero invece misure di latenza e numero di immagini elaborate per unità di tempo, distinte dai tempi di addestramento qui riportati.

4.4.4 CCW-SSIM e il ruolo dell'informazione cromatica

La fonte di Berasi [3] ha come obiettivo centrale l'integrazione dell'informazione cromatica nella misura strutturale utilizzata da un autoencoder. Il confronto tra varianti della funzione di perdita è quindi essenziale per valutarne il contributo. Il presente lavoro include CCW-SSIM nel confronto con tre pipeline basate su feature, ma non contiene l'ablazione controllata rispetto alla variante CW-SSIM. Non può pertanto confermare né confutare direttamente il vantaggio della componente cromatica.

Un'ulteriore distinzione riguarda le metriche. La Tabella 1 della fonte riporta per `leather` il valore 0,982 relativo alla curva ROC; nella sezione sulle prestazioni, la fonte specifica una normalizzazione dell'area limitata a FPR pari a 0,3. La nostra AUROC pixel, pari a 0,9707, integra invece la curva completa. Anche l'AU-PRO locale non è la stessa misura, perché integra la copertura delle regioni anziché il tasso di veri positivi pixel. Nessuno dei due valori consente quindi un confronto diretto con 0,982.

Il risultato locale di CCW-SSIM va dunque letto rispetto alla pipeline utilizzata. La fonte distingue il beneficio cromatico all'interno dell'approccio ricostruttivo dal confronto con metodi più sofisticati dello stato dell'arte. Per approfondire tale beneficio nel progetto occorre mantenere fissi architettura, dati e valutatore e variare la componente della perdita.

La verifica del percorso effettivo individua inoltre differenze nella componente cromatica, esplicitate nel Capitolo 3: errore quadratico anziché distanza euclidea, aggregazione globale durante l'addestramento e coefficienti differenti in inferenza. Queste scelte preservano l'idea di combinare struttura e colore, ma non permettono di trasferire automaticamente alla variante locale il beneficio cromatico dimostrato dalla fonte.

Per approfondire le scelte di implementazione, un test diagnostico del checkpoint esistente, senza addestramento e senza sostituire le mappe ufficiali, ha elaborato dodici patch 128×128 estratte da quattro posizioni fissate nelle prime tre immagini normali di test. In tali patch la luminanza ricostruita è già positiva. L'RMSE sul canale L^* è compreso tra 2,08 e 7,03 unità Lab prima dell'inversione e tra 63,90 e 72,02 dopo. L'inversione non è dunque giustificata come correzione necessaria in questi campioni. Non cambia però la perdita CW-SSIM misurata, che rimane pari a 0,153519: il modulo del prodotto dei coefficienti rende la misura insensibile all'inversione globale del segno. La componente cromatica usa soltanto a^* e b^*. L'inversione del canale L^* non spiega pertanto il divario nelle metriche della mappa.

Nel medesimo test, la perdita totale è 0,153527, superiore alla componente strutturale di circa $8{,}43 \times 10^{-6}$. La correzione cromatica media prima della moltiplicazione è circa $9{,}96 \times 10^{-6}$ durante l'addestramento; il coefficiente di inferenza darebbe circa 0,00448 prima della media spaziale. Questi valori accertano una piccola incidenza cromatica sul valore della perdita in queste patch e una diversa pesatura in inferenza, non dimostrano l'effetto sui gradienti o sulle prestazioni. Il test non sostituisce un'ablazione.

Una verifica complementare riguarda la posizione dei picchi che determinano gli errori di classificazione. L'analisi spaziale delle mappe archiviate localizza il massimo di otto dei nove falsi positivi entro tre pixel dal bordo. Il campione `good/030.png`, già discusso, ha invece il massimo all'interno dell'immagine. Ciò individua una criticità del punteggio aggregato mediante massimo e della risposta periferica, senza provare quale componente la generi. Non viene applicata una correzione dei bordi scelta sulle etichette del test; i risultati principali conservano la mappa originale del protocollo originale.

La previsione su patch da 256×256 con stride 16 è allineata alla procedura descritta dalla fonte. Il riferimento a 50 000 patch normali di addestramento non va invece equiparato ai 49 000 aggiornamenti locali: un aggiornamento utilizza un batch di patch e le due quantità misurano aspetti diversi dell'esperimento. Le mappe e gli errori analizzati offrono una spiegazione descrittiva del risultato locale, mostrando risposte alla texture normale e copertura parziale di alcuni difetti. Non permettono di stabilire perché la prestazione differisca da quella della pubblicazione, prima ancora di rendere omogenee le metriche.

4.4.5 Interpretazione degli scostamenti

Per interpretare gli scostamenti, distinguo tre livelli di evidenza. Il primo è il risultato misurato: per PaDiM gli scarti sono piccoli, per RD4AD emerge soprattutto una differenza nel PRO e per FastFlow le AUROC sono in linea

o lievemente superiori ai valori riportati. Il secondo comprende le differenze documentate, come il preprocessing di PaDiM, il regime di addestramento di FastFlow e la metrica della fonte CCW-SSIM. Il terzo comprende le ipotesi esplicative, tra cui la sensibilità del PRO ai bordi e la variabilità delle esecuzioni. Soltanto i primi due livelli sono verificati dal materiale disponibile; per il terzo servono prove dedicate.

La consistenza del confronto interno è sostenuta anche dal ricalcolo completo delle mappe archiviate: per ciascuna pipeline sono confermate le tre aree principali e tredici valori aggiuntivi relativi a F1, soglie, precisione, richiamo e scala dei punteggi, senza differenze numeriche rispetto ai file di risultato. Questa verifica collega le metriche ai dati e al valutatore comuni. Il rapporto con le fonti rimane invece condizionato dalle differenze documentate: il protocollo regionale di RD4AD, le condizioni di apprendimento e le misure di efficienza di FastFlow e la variante cromatica locale di CCW-SSIM.

4.5 Limiti e portata dei risultati

La scelta di un unico materiale consente di approfondire cinque tipologie di difetto, ma circoscrive anche le conclusioni a `leather`. Per estenderle a materiali e oggetti differenti sarebbe necessaria una valutazione su altre categorie.

La stabilità delle differenze osservate richiede inoltre repliche. Una sola esecuzione per pipeline non quantifica la variabilità rispetto al seme pseudocasuale o all'addestramento e non permette di stabilire una superiorità statisticamente significativa. Anche i pixel di una stessa immagine sono correlati: la loro numerosità non equivale a un insieme altrettanto ampio di osservazioni indipendenti.

Il passaggio a un impiego operativo dipende dalla scelta delle soglie e dalla composizione dei dati. Le soglie ottimali sono selezionate sul test stesso e le immagini difettose sono 92 su 124, una proporzione che non descrive necessariamente una linea produttiva. Precisione e F1 possono cambiare al variare della prevalenza delle anomalie. Una valutazione operativa richiederebbe soglie definite su dati separati e misure coerenti con il costo delle omissioni e dei falsi allarmi.

Infine, l'analisi qualitativa si basa su pochi campioni scelti con criteri espliciti. I conteggi per tipologia riguardano la classificazione delle immagini; non sono disponibili in questa analisi metriche di localizzazione separate per ciascun tipo di difetto. Le differenze di pre-addestramento, risoluzione e post-elaborazione impediscono inoltre una spiegazione causale delle differenze osservate. Questi limiti non annullano il confronto, ma ne delimitano le conclusioni.

4.6 Discussione complessiva

Il primo risultato da considerare è la separazione completa delle immagini normali e difettose ottenuta da PaDiM, RD4AD e FastFlow nel caso di studio.

Per confrontare i tre modelli occorre però guardare anche alla localizzazione: FastFlow ottiene i valori più alti delle metriche considerate, RD4AD segue con prestazioni prossime e PaDiM mantiene risultati elevati con il costo di adattamento più contenuto. Le mappe aiutano a interpretare questa distinzione: il riconoscimento corretto dell'immagine può convivere con risposte più estese del difetto.

CCW-SSIM presenta un costo di apprendimento maggiore, una separazione delle immagini meno efficace e una copertura regionale inferiore. Individua comunque 84 delle 92 immagini difettose alla soglia di massimo F1. Le omissioni e il campione normale analizzato indicano che la relazione tra risposta alla texture, aggregazione del punteggio e localizzazione merita attenzione, senza che gli esperimenti disponibili consentano di isolare il contributo di ciascun fattore.

Il confronto con le fonti permette di leggere questi risultati nel contesto delle configurazioni e delle metriche adottate. Le differenze documentate delimitano il significato del riscontro numerico; per spiegare le cause degli scarti occorrono esperimenti dedicati. A partire da queste osservazioni, nel Capitolo 5 torno agli obiettivi iniziali e discuto gli approfondimenti necessari per sviluppare il lavoro.

Capitolo 5

Conclusioni e sviluppi futuri

5.1 Bilancio del lavoro

Il confronto tra CCW-SSIM, PaDiM, RD4AD e FastFlow mi ha permesso di esaminare come quattro approcci al rilevamento delle anomalie visive industriali descrivano la normalità e riconoscano i difetti. Il passaggio dai principi dello stato dell'arte alle configurazioni adottate si è basato sulle immagini della categoria `leather` di MVTec AD e su un protocollo comune di valutazione. Alle metriche ho affiancato l'esame delle mappe di anomalia e dei casi di errore, per considerare insieme il riconoscimento delle immagini difettose e la localizzazione delle regioni anomale.

PaDiM, RD4AD e FastFlow hanno separato completamente le immagini normali e difettose del test esaminato. Per valutarne le differenze è stato necessario considerare anche la localizzazione: FastFlow ha ottenuto i valori più elevati delle metriche considerate, seguito da RD4AD e PaDiM. Nelle mappe una risposta nella posizione corretta può estendersi oltre il contorno annotato; precisione e richiamo descrivono il compromesso tra copertura del difetto e segnalazione di pixel normali. Da questa lettura emerge la necessità di valutare il risultato a livello di immagine insieme alla qualità della localizzazione.

CCW-SSIM ha presentato una separazione delle immagini meno efficace e una copertura regionale inferiore. Negli esempi analizzati, le risposte alla texture normale rendono meno netto il contrasto del difetto e possono contribuire a una segnalazione anche in assenza di anomalie. L'esame dei casi visivi è stato quindi utile per comprendere aspetti del comportamento della pipeline che la sola AUROC non descrive pienamente.

Anche i tempi di apprendimento riflettono le differenze tra gli approcci. PaDiM ha richiesto il minor tempo di adattamento locale, poiché stima le statistiche delle feature senza aggiornare l'estrattore. CCW-SSIM ha richiesto il tempo maggiore, mentre RD4AD e FastFlow si sono collocati in posizione intermedia. Le misure descrivono la preparazione delle pipeline nelle condizioni adottate; il

costo del pre-addestramento esterno e le risorse necessarie durante l'inferenza richiedono valutazioni distinte.

Il confronto con le pubblicazioni ha mostrato AUROC di PaDiM, RD4AD e FastFlow prossime ai valori riportati per `leather`. Nel valutarne il significato, considero però anche le differenze di configurazione e valutazione. Per RD4AD cambia il protocollo di calcolo del PRO; per CCW-SSIM differiscono la metrica della fonte e la formulazione della componente cromatica. Le verifiche diagnostiche hanno approfondito alcuni aspetti della variante locale, senza isolare il beneficio del colore rispetto a CW-SSIM. La valutazione di tale contributo resta dunque una domanda aperta.

Ritengo che il contributo del lavoro risieda nel confronto documentato tra prestazioni, costi di adattamento e comportamento delle mappe. La registrazione di dati, configurazioni e risultati permette di ricondurre le osservazioni alle esecuzioni analizzate. Questa impostazione ha consentito di riconoscere, nel caso di studio, sia le capacità delle diverse pipeline sia i punti che richiedono un approfondimento sperimentale.

5.2 Portata delle conclusioni

I risultati riguardano un unico materiale e le configurazioni utilizzate nel progetto. Le cinque tipologie di difetto permettono di esaminare manifestazioni diverse delle anomalie all'interno di `leather`, mentre la generalizzazione ad altri materiali richiede ulteriori esperimenti. Le differenze di risoluzione, preaddestramento e trattamento delle mappe sono parte del confronto e non sono state isolate mediante prove controllate.

Una sola esecuzione per modello non permette inoltre di quantificare la variabilità dei risultati. I piccoli scarti tra pipeline descrivono le esecuzioni osservate, ma non dimostrano una superiorità statisticamente significativa. Anche la scelta delle soglie delimita il significato delle prestazioni: il massimo F1 è calcolato sul test stesso e la proporzione di immagini difettose può differire da quella di una linea produttiva. Il passaggio all'impiego industriale richiederebbe soglie selezionate su dati separati e una valutazione del compromesso tra omissioni e falsi allarmi in funzione dell'applicazione.

Alla luce di questi limiti, considero prioritario approfondire i componenti delle pipeline, verificare la stabilità delle prestazioni e valutare il comportamento in condizioni operative differenti.

5.3 Sviluppi futuri

Un primo punto che approfondirei è il contributo dell'informazione cromatica in CCW-SSIM. La diversa pesatura tra addestramento e inferenza suggerisce una revisione controllata della componente cromatica, con parametri coerenti con la normalizzazione. Il confronto con CW-SSIM, mantenendo invariati architettura, dati, addestramento e valutatore, permetterebbe poi di misurare il beneficio

del colore. All'analisi della tipologia `color` andrebbe affiancata quella degli altri difetti e delle risposte sulla texture normale. La modifica della perdita richiederebbe un nuovo addestramento e risultati identificati come un esperimento successivo.

La stabilità del confronto potrebbe essere esaminata attraverso repliche con diversi semi pseudo-casuali. All'interno di `leather`, il calcolo delle metriche di localizzazione per tipologia di difetto completerebbe inoltre i conteggi di classificazione già disponibili. Il confronto con i valutatori originali, soprattutto per il PRO di RD4AD, aiuterebbe a distinguere le differenze di calcolo da quelle delle pipeline.

Un altro sviluppo riguarda la scelta del punto operativo. Un insieme di validazione separato consentirebbe di definire le soglie prima della valutazione sul test. Per CCW-SSIM, il confronto tra il massimo della mappa e aggregazioni alternative permetterebbe di verificare se una minore sensibilità ai picchi isolati riduca i falsi positivi mantenendo il rilevamento dei piccoli difetti.

La valutazione applicativa richiederebbe infine misure di latenza, numero di immagini elaborate per unità di tempo e memoria di inferenza sullo stesso hardware, distinguendo preprocessing, esecuzione del modello e post-elaborazione. Prove controllate con variazioni di illuminazione e colore permetterebbero di esaminare la robustezza alle condizioni di acquisizione; l'estensione ad altri materiali consentirebbe di studiare la generalizzazione.

Il punto che emerge dal caso di studio è l'utilità di valutare la capacità di riconoscere un'immagine difettosa insieme alla localizzazione, alle soglie e ai costi osservati. I metodi basati su feature hanno ottenuto i risultati più favorevoli nelle configurazioni adottate. Le domande rimaste aperte indicano da dove proseguire per verificarne la stabilità e approfondire il comportamento dei modelli in nuovi esperimenti.

Bibliografia

[1] Abdelrahman Alzarooni, Ehtesham Iqbal, Samee Ullah Khan, Sajid Javed, Brain Moyo, and Yusra Abdulrahman. Anomaly detection for industrial applications, its challenges, solutions, and future directions: A review. *arXiv preprint arXiv:2501.11310*, 2025.

[2] Aimira Baitieva, David Hurych, Victor Besnier, and Olivier Bernard. Supervised anomaly detection for complex industrial images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17754–17762, 2024.

[3] Simone Berasi. Ccw-ssim: a novel color-enhanced structural similarity metric applied to autoencoders for anomaly detection. Master's thesis, Politecnico di Milano, 2023.

[4] Paul Bergmann, Kilian Batzner, Michael Fauser, David Sattlegger, and Carsten Steger. The mvtec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. *International Journal of Computer Vision*, 129(4):1038–1059, 2021.

[5] João P. C. Bertoldo, Dick Ameln, Ashwin Vaidya, and Samet Akçay. Aupimo: Redefining anomaly localization benchmarks with high speed and low tolerance. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2024.

[6] Yunkang Cao, Xiaohao Xu, Jiangning Zhang, Yuqi Cheng, Xiaonan Huang, Guansong Pang, and Weiming Shen. A survey on visual anomaly detection: Challenge, approach, and prospect. *arXiv preprint arXiv:2401.16402*, 2024.

[7] Yajie Cui, Zhaoxiang Liu, and Shiguo Lian. A survey on unsupervised anomaly detection algorithms for industrial images. *IEEE Access*, 11:55297–55315, 2023.

[8] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. Padim: a patch distribution modeling framework for anomaly detection and localization. In *Pattern Recognition. ICPR International Workshops and Challenges, Part IV*, volume 12664 of *Lecture Notes in Computer Science*, pages 475–489. Springer, 2021.

[9] Hanqiu Deng. RD4AD: implementazione ufficiale, commit 6554076. `https://github.com/hq-deng/RD4AD/tree/6554076872c65f8784f6ece8cfb39ce77e1aee12`, 2022. File test.py e dataset.py; consultati il 2 ottobre 2026.

[10] Hanqiu Deng and Xingyu Li. Anomaly detection via reverse distillation from one-class embedding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9737–9746, 2022.

[11] Kamil Faber, Christopher Kanan, Vincenzo Lomonaco, and Roberto Corizzo. Continual anomaly detection: A comprehensive survey and research roadmap. *Preprints.org*, 2026. Preprint, versione 1; non sottoposto a peer review.

[12] Lars Heckler-Kram, Jan-Hendrik Neudeck, Ulla Scheler, Rebecca König, and Carsten Steger. The mvtec ad 2 dataset: Advanced scenarios for unsupervised anomaly detection. *International Journal of Computer Vision*, 134:175, 2026. Articolo 175.

[13] Anton Korinek. Language models and cognitive automation for economic research. Working Paper 30957, National Bureau of Economic Research, 2023.

[14] Tangqing Li, Zheng Wang, Siying Liu, and Wen-Yan Lin. Deep unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 3636–3645, 2021.

[15] Zhuo Li, Yuhao Yan, Xiangheng Wang, Yifei Ge, and Lin Meng. A survey of deep learning for industrial visual anomaly detection. *Artificial Intelligence Review*, 58(9):279, 2025.

[16] Roya Morshedi and S. Mojtaba Matinkhah. A comprehensive review of deep learning techniques for anomaly detection in iot networks: Methods, challenges and datasets. *Engineering Reports*, 7(9):e70415, 2025.

[17] MVTec. The mvtec logical constraints anomaly detection dataset. Consultato il 2 ottobre 2026.

[18] G. Pang, C. Shen, L. Cao, and A. Van Den Hengel. Deep learning for anomaly detection: A review. *ACM Computing Surveys (CSUR)*, 54(2):1–38, 2021.

[19] Guansong Pang, Longbing Cao, and Charu Aggarwal. Deep learning for anomaly detection: Challenges, methods, and opportunities. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 1127–1130, 2021.

[20] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Scholkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14318–14328, 2022.

[21] Thomas Schlegl, Philipp Seeböck, Sebastian M. Waldstein, Ursula Schmidt-Erfurth, and Georg Langs. Unsupervised anomaly detection with generative adversarial networks to guide marker discovery. *arXiv preprint arXiv:1703.05921*, 2017.

[22] Tung Minh Tran, Tu N. Vu, Nguyen D. Vo, Tam V. Nguyen, and Khang Nguyen. Anomaly analysis in images and videos: A comprehensive review. *ACM Computing Surveys*, 55(7):148:1–148:37, 2022.

[23] Miryam Elizabeth Villa-Pérez, Miguel Á. Álvarez-Carmona, Octavio Loyola-González, Miguel Angel Medina-Pérez, Juan Carlos Velazco-Rossell, and Kim-Kwang Raymond Choo. Semi-supervised anomaly detection algorithms: A comparative summary and future research directions. *Knowledge-Based Systems*, 218:106878, 2021.

[24] Y. Wan, Y. Lang, and L. Yao. Dcs: A zero-shot anomaly detection framework with dino-clip-sam integration. *Applied Sciences*, 16(4):1836, 2026.

[25] Jie Yang, Ruijie Xu, Zhiquan Qi, and Yong Shi. Visual anomaly detection for images: A survey. *arXiv preprint arXiv:2109.13157*, 2021.

[26] Jiawei Yu, Ye Zheng, Xiang Wang, Wei Li, Yushuang Wu, Rui Zhao, and Liwei Wu. Fastflow: Unsupervised anomaly detection and localization via 2d normalizing flows. *arXiv preprint arXiv:2111.07677*, 2021.

[27] Vitjan Zavrtanik, Matej Kristan, and Danijel Skočaj. DRAEM: A discriminatively trained reconstruction embedding for surface anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8330–8339, 2021.

[28] Ye Zheng, Xiang Wang, Yu Qi, Wei Li, and Liwei Wu. Benchmarking unsupervised anomaly detection and localization. *arXiv preprint arXiv:2205.14852*, 2022.